Neural Computing and Applications (2024) 36:1067–1089
https://doi.org/10.1007/s00521-023-09078-8

ORIGINAL ARTICLE

End-to-end acceleration of the YOLO object detection framework on FPGA-only devices

Dezheng Zhang[1,2] · Aibin Wang[1,2] · Ruchan Mo[1,2] · Dong Wang[1,2]

Received: 23 December 2022 / Accepted: 14 September 2023 / Published online: 13 November 2023
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2023

Abstract

Object detection has been revolutionized by convolutional neural networks (CNNs), but their high computational complexity and heavy data access requirements make implementing these algorithms on edge devices challenging. To address this issue, we propose an efficient object detection accelerator for YOLO series algorithm. Our architecture utilizes multiple dimensions of parallelism to accelerate the convolution computation. We employ line-buffer-based parallel data caches and dedicated data access units to minimize off-chip bandwidth pressure. Additionally, our proposed design not only accelerates the convolutional computation, but also control-intensive post-processing to achieve low detection latency. We evaluate the final design on Xilinx V7-690t FPGA device, achieving a throughput of 525 GOP/s for a batch size of 1 and 914 GOP/s for a batch size equal to 2. Compared with state-of-the-art YOLOv2 and YOLOv3 implementations, our proposed accelerator offers up to 9× throughput improvement and 5× shorter latency.

Keywords Convolution neural networks (CNN) · Object detection · YOLOv2 · Field-programmable gate array (FPGA) · High-level synthesis (HLS) · Accelerator architecture · Post-processing

1 Introduction

Deep learning-based object detection algorithms have been applied in many fields, such as autonomous driving [1], medical image detection [2], and industrial inspection [3]. However, these algorithms still face some crucial problems, such as the high computational workload and bandwidth requirement of the convolutional neural network (CNN) computation, which restricts the actual application. Although a few object detection accelerators have been proposed, they still face some challenges. In the approach [4, 5], two streaming object detection accelerators are designed, which map all the CNN layers within the YOLOv2-Tiny on a field programmable gate array (FPGA). All nine convolution layers are pipelined, which reduces the off-chip data access and improves the throughput. Though low latency and high digital signal processor (DSP) efficiency are achieved, it is challenging to implement very deep CNN algorithms such as YOLOv2 or YOLOv3 on them because the number of layers is restricted by the hardware resource. In the approach [6], an implementation of YOLOv3-Tiny is proposed, which achieves 460GOPS/s on the VC707 development board with a data width of 18-bit. Similarly, the work in the literature [7] introduced fast Fourier transform (FFT) to speed up the convolution operation. However, feature maps and weights are represented by relatively high bit width data types, i.e., 16-bit fixed-point numbers. This causes a higher consumption of computational and memory resources. In contrast, most CNN models can be represented by 8-bit data with negligible loss of accuracy, according to recent quantization research [8]. Lower data bit width can reduce DSP and memory consumption on

✉ Dong Wang
wangdong@bjtu.edu.cn

Dezheng Zhang
dezhengzhang@bjtu.edu.cn

Aibin Wang
21120301@bjtu.edu.cn

Ruchan Mo
moruchan@bjtu.edu.cn

[1] Institute of Information Science, Beijing Jiaotong University, Beijing 100044, China

[2] Beijing Key Laboratory of Advanced Information Science and Network Technology, Beijing 100044, China

FPGA, which enables the design to achieve equivalent performance with fewer hardware resources and lower energy consumption.

Furthermore, in most accelerator designs [4–6, 9], the pre- and post-processing algorithms, such as the softmax layer, intersection-over-union (IOU) calculation, and non-maximum suppression (NMS), are typically implemented on central processing units (CPUs). However, in certain scenarios with stringent latency requirements, CPU-based implementation schemes can often fail to satisfy the low latency demands, as indicated by studies of [10, 11]. For instance, in industrial robotics, detecting targets within a brief latency window is crucial. This imposes strict requirements not just on the CNN computational latency but also on the post-processing latency. Therefore, it becomes necessary to devise an object detection accelerator that incorporates a dedicated hardware unit for post-processing algorithms.

In this work, an end-to-end YOLO-based object detection hardware accelerator targeting implementation on FPGA fabric-only devices is proposed. The accelerator is designed with a flexible architecture in which each layer of the convolutional neural networks are computed in a sequential way so that it is possible to implement object detection algorithms with different sizes, e.g., YOLOv2 and YOLOv3. However, this structure increases the difficulty of data accessing and caching. To address this challenge, three data access units and an on-chip buffer are designed, where the former utilize the burst transfer to fully take advantage of the off-chip bandwidth, and the latter improves the on-chip data reuse. Moreover, the data access is overlapped with the convolution computation. The proposed accelerator is finally implemented and evaluated on the Xilinx's V7-690t FPGA device showing a throughput improvement of up to 9× compared to the previous implementations [4, 6, 9] for YOLOv2 or YOLOv3. Furthermore, the pre- and post-processing of the algorithm are also conducted on the FPGA fabric, which further decreases the detection latency.

In summary, the main contributions of this paper are summarized as follows.

(1) A novel end-to-end object detection accelerator targeting FPGA-only devices is proposed. The proposed architecture is scalable in multiple computational dimensions, enabling efficient acceleration of complex object detection neural network models, such as YOLOv2 and YOLOv3, on devices with varying resource constraints. In addition, FPGA fabrics are utilized to expedite post-processing operations in the object detection algorithms, thereby minimizing detection latency. Our design offers a unique approach to object detection acceleration, which addresses the challenges of deploying large network models on FPGA-only devices.

(2) An on-chip data caching strategy is proposed, which overlaps the data transfer time with convolutional computation, resulting in improved execution efficiency of the accelerator. The proposed approach utilizes a ping-pong buffer to enhance the data reuse of feature maps and weights. Additionally, weight-loading unit and feature loading unit are designed to operate independently. Their state synchronization occurs only when a computation group's workload is complete, reducing the synchronization frequency and minimizing accelerator stall.

(3) The proposed accelerator is implemented in parameterized HLS code, allowing for quantitative performance and hardware resource consumption modeling. A design space exploration scheme is developed to facilitate an automatic and fast search of the most appropriate parameter setting for any specific FPGA device.

The rest of the paper is organized as follows. Section 2 gives the background, discussing the structure of the YOLO series algorithms and the quantization method used in this paper. Section 3 presents the accelerator design, including data access unit design, computation unit design, and post-processing unit design. Section 4 details the modeling of the accelerator's performance and resource consumption and provides the optimal design parameters based on these models. The experiment setup is described in Sect. 5, which also discusses the experiment results. Finally, Sect. 6 concludes the work. For ease of reading, the abbreviations and symbols used in this paper are summarized in Table 1.

2 Detection algorithm preparation

In this section, the structure of the YOLO series algorithms is briefly introduced. Then, batch normalization (BN) fusion is applied. Finally, the activation and the weights are quantized to 8-bit.

2.1 Detection algorithm overview

As shown in Fig. 1, the YOLO series algorithms are composed of preprocessing, CNN computation, and post-process procedures. In the preprocessing stage, the input is resized to a uniform size and then normalized. The CNN backbone neural network consists of the most computation workload in the inference procedure. Additionally, leaky ReLU and BN follow the convolution layers. The input of the CNN network is divided into $D \times D$ grids, and a total of

Table 1 Abbreviations

Symbol	Description
B	Word length of the fixed-point number
fl	Fractional length of the fixed-point number
N_r	Number of CUs within one PE array
N_p	Number of PEs within one CU
$N_n \times 2$	Number of MACs performed by one PE in parallel
N_b	Number of PE arrays
N_m	Parallelism along the channel direction of the output
$2N_{RAM}$	Number of the line buffers within one feature buffer
D_f	The depth of the feature buffer
N	Number of channels of the input feature map
R	Number of rows of the input feature map
C	Number of columns of the input feature map
M	Number of channels of the output feature map
R'	Number of rows of the output feature map
C'	Number of columns of the output feature map
K	Number of rows of the convolution kernel
K'	Number of columns of the convolution kernel
S	Stride of the convolution
Q_l	Width of the computation group for the l-th layer
G_l^r	Number of computation groups in the row direction
G_l^c	Number of computation groups in the column direction
G_l^m	Number of computation groups in the channel direction
H_{win}	Height of the pre-fetch window
W_{win}	Width of the pre-fetch window
T_l	Theoretical inference latency of the l-th layer
C_1	Number of DSPs consumed except PE arrays
C_2	Factor measuring the BRAM used by weight buffer
C_3	Factor measuring the BRAM used by feature buffer
C_4	Amount of BRAM consumed except the local buffer
C_5	Factor measuring the LUT used by an adder tree
C_6	Factor measuring the LUT used by an adder tree
C_7	Amount of logic consumption of a 32-bit accumulator
C_8	Amount of logic consumed except the PE arrays

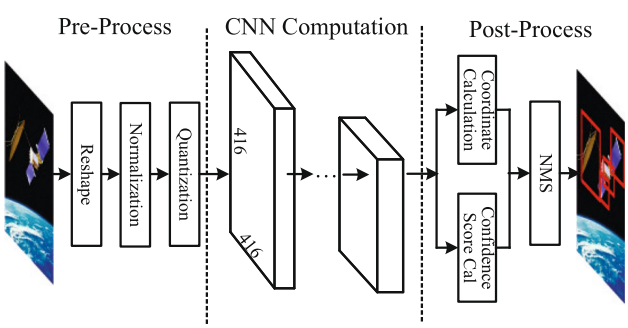

Fig. 1 Architecture of the YOLO network

P vectors are generated by each grid. Each vector corresponds to a predicted bounding box, and it is composed of $(5 + C)$ numbers where one number corresponds to the probability of containing an object, four numbers are responsible for the size and location of the bounding box, and the rest C numbers correspond to the category information. So, the output of size $D \times D \times P \times (5 + C)$ is generated after convolution operations.

In the post-processing procedure, coordinates and confidence scores are calculated using the CNN outputs. Post-processing consists of computation-intensive operations and control-intensive operations. Most of the computational resource is consumed by the sigmoid and exponential operations when decoding the coordinates of the bounding boxes. Then, NMS is carried out to filter out the redundant predictions, which is a control-intensive operation. Finally, the predicted objects are shown to the users.

2.2 Network structure optimization

Batch normalization helps to accelerate the training and prevent overfitting [12] when training the neural network. During the inference of the neural network, the BN operation is denoted as

$$\hat{y_m} = \gamma_m \times \frac{y_m - \mu_m}{\sqrt{\sigma_m^2 + \epsilon}} + \beta_m \tag{1}$$

where y_m and $\hat{y_m}$ denote the input and output of the BN operation, respectively, and the subscript m denotes channel index. The fixed estimation of mean and variance of the mini-batch is denoted as μ_m and σ_m^2, respectively, and the γ_m and β_m are learnable parameters. Moreover, the ϵ, generally equally 1.0×10^{-5}, is added to the denominator for numerical stability. Batch normalization helps to achieve a stable distribution of activation, but the multiplication and division consume a lot of hardware resource when the network is implemented on the FPGA. Batch normalization can be seen as a linear operation like $\hat{y_m} = \alpha_m \times y_m + \eta_m$ where

$$\alpha_m = \frac{\gamma_m}{\sqrt{\sigma_m^2 + \epsilon}}, \tag{2}$$

$$\eta_m = \beta_m - \frac{\mu_m \gamma_m}{\sqrt{\sigma_m^2 + \epsilon}}. \tag{3}$$

Taking BN and convolution calculation into account, the process can be denoted as

$$\hat{y_m} = \alpha_m \times (x * W_m + B_m) + \eta_m, \tag{4}$$

where x, W_m and B_m denote the input, weight and bias of the convolution computation. The linear operation in Eq. 4 can be further fused into the convolution calculation, i.e.,

$$\hat{y_m} = x * (\alpha_m \times W_m) + (\alpha_m \times B_m + \eta_m), \tag{5}$$

where $\alpha_m \times W_m$ and $\alpha_m \times B_m + \eta_m$ are the updated weight and bias.

Furthermore, in the original network, the location and confidence are extracted by the neural network within one layer, for example, the 23rd filter in the YOLOv2 algorithm. A total of 100 images are randomly selected as the input of the network, and their distribution of the location and category information is shown in Fig. 2. It can be seen that the location information is mainly distributed between -2 and 2, while the category information is mainly distributed between -15 and 10. The difference in the numerical distribution makes it difficult to do the model quantization. Therefore, that convolutional layer is divided into two independent convolution layers, where one extracts the location information, and the other is responsible for the category information, and they are quantized separately.

2.3 Network quantization

Previous studies have shown that the parameters of neural networks can be represented by 8-bit dynamic fixed-point numbers without introducing obvious accuracy loss [13, 14]. Neural network quantization not only relieves the pressure on the memory system, but also improves the efficiency of the DSP. Specifically, compared to a full precision network, the storage and bandwidth requirement of the 8-bit quantized model decreases by four times [15]. In current Intel and Xilinx FPGA devices [4, 16], two 8×8 fixed-point multiplication can also be efficiently implemented by consuming only one DSP block. This reduces hardware consumption and enables more operations to be carried out concurrently. In this work, the model and the activation are represented using the 8-bit dynamic fixed-point number [15]. The value x of a B-bit fixed-point number $x_{B-1}x_{B-2}\ldots x_0$ with the fractional length fl is given by

$$x = \left(-x_{B-1} \cdot 2^{B-1} + \sum_{i=0}^{B-2} x_i \cdot 2^i\right) \cdot 2^{-fl}. \tag{6}$$

The fl correlates with the step size or resolution, and the representation range of a dynamic fixed-point number. To cover the large dynamic range of the data in CNN inference, the input, output, weight, and bias of each layer are quantized, respectively [15]. However, the parameter fl is an integer, and it is challenging to be optimized using back-propagation. In this work, we calculate the fl parameter from a continuous step size r using the equation $fl = -\lfloor log_2^r \rfloor$. The parameter r is calculated using the BRECQ quantization method [13].

In the BRECQ scheme, the quantization is performed on the weights, biases, and activations. For each layer in the CNN, the step size r for weights and biases is initialized by exhaustive search to minimize the quantization loss:

$$r_w^* = \arg\min_{r_w} \|\hat{w}_{init} - w\|_F^2, \tag{7}$$

where $\|\cdot\|_F$ denotes the Frobenius norm, w denotes the weights or bias before the quantization, r_w^* denotes the initialized step size, and $\hat{w}_{init}$ denotes the initial quantized weights or bias which equals to

$$\hat{w}_{init} = r_w \times clip(\lfloor w/r_w \rfloor, -2^{B-1}, 2^{B-1} - 1). \tag{8}$$

To reduce the quantization loss, a learnable parameter v is added to the $\hat{w}_{init}$ to calibrate the least significant bits (LSBs) of the weights or bias. A sigmoid-like function $\sigma(\cdot)$ is then used to ensure that $\sigma(v)$ equals to 0 or 1. With this calibration, the final weight $\hat{w}$ is equal to

$$\hat{w} = r_w^* \times clip(\lfloor w/r_w^* \rfloor + \sigma(v), -2^{B-1}, 2^{B-1} - 1). \tag{9}$$

In addition, a small number of randomly selected images

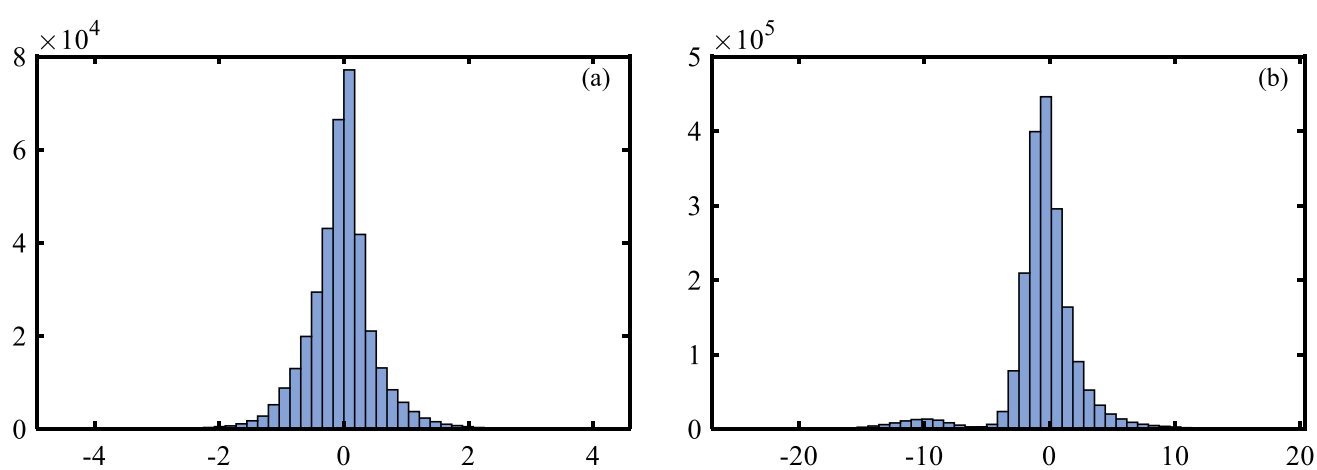

Fig. 2 Distribution of the location and category data. **a** The distribution of the location data. **b** The distribution of the category data

are fed into the neural network as inputs. The difference between the convolution result of the quantized weight and the convolution result of the original weight is taken as the loss function. Afterward, back-propagation is used to optimize the learnable parameter v and the step size r_w, i.e.,

$$v, r_w = \arg\min_{v,r_w} \|x * w - x * \hat{w}\|_F^2. \tag{10}$$

Similar to the weight quantization, the quantized activation $\hat{x}$ is equals to

$$\hat{x} = r_x \times clip(\lfloor x/r_x \rfloor, -2^{B-1}, 2^{B-1} - 1), \tag{11}$$

where the step size r_x is also initialized and optimized using the similar method with the weight quantization.

3 Accelerator hardware design

The top-level architecture of the proposed accelerator is illustrated in Fig. 3. The proposed hardware accelerator is mainly composed of central processing element (PE) arrays, feature load unit, weight load unit, feature store unit, task scheduler unit, max-pooling unit and pre/post-processing unit. A MicroBlaze processor is responsible for sending coarse-grained instruction to control the accelerator. The feature load unit and weight load unit are responsible for loading the features and weights from the external double data rate (DDR) synchronous dynamic random-access memory (SDRAM) to the on-chip buffers, which are designed to cache the required data for convolution operations to improve the data reuse and external memory bandwidth efficiency. The PE array is designed to accelerate the neural network computation by utilizing as many on-chip DSP blocks as possible to perform parallel convolution operations. Batch processing is also introduced by instancing multiple PE arrays to further improve the throughput. The convolution results are then written back to the external memory through the feature store unit. Each function unit has its own loop counter, and synchronizations between the functional units are controlled by the Task Scheduler unit. Synchronizations are only performed when all the cached data have been consumed by the PE array. The low synchronization frequency reduces the risk of accelerator stall caused by the high-latency off-chip memory access. Moreover, the preprocessing and post-processing algorithms are done by the dedicated hardware unit.

3.1 PE array design

As shown in Fig. 3, the PE array is organized as N_r compute units (CUs). Each CU, consisting of N_p PEs, performs

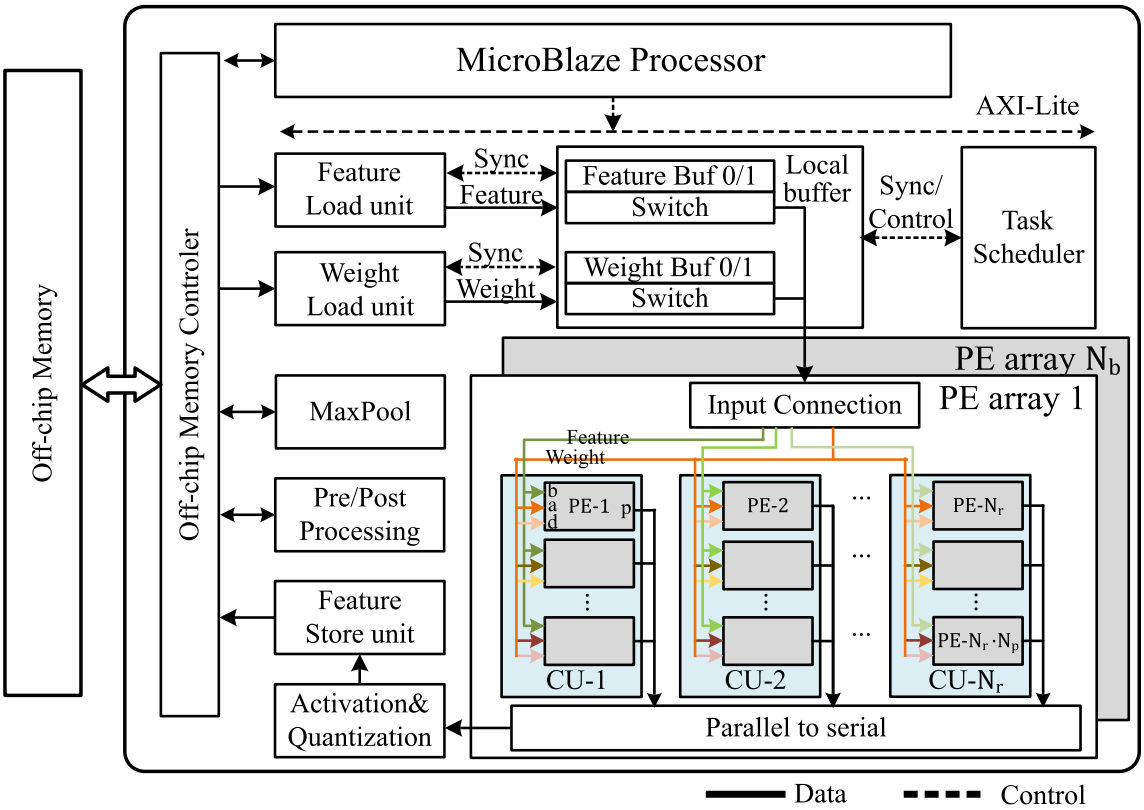

Fig. 3 Architecture of the accelerator

the multiply-accumulate operations (MAC) involved in the convolution neural network algorithm in parallel. The proposed multi-CU architecture is designed to relieve the stress in place and route caused by the high fan-out of the PE array and improve the scalability of the overall design. Those CUs are connected to the input connection and output connection interfaces. Three vector input ports ($\vec{a}$, $\vec{b}$, and $\vec{d}$) and one output port ($\vec{p}$) are designed for each PE, in which two inner product operations are conducted concurrently, i.e., $\vec{b} \cdot \vec{a}$ and $\vec{b} \cdot \vec{d}$. The two inner products share a common input vector $\vec{b}$, which can reduce the complexity of the on-chip data busses and improve the arithmetic intensity of the executed computation. Moreover, this operation pattern could also be utilized to improve the peak performance of the convolution computation by up to 2× with the same on-chip DSP cost by packing the two multiplication into one DSP block [4].

The detailed architecture of the PE is shown in Fig. 4a. Each PE contains N_n DSP48 blocks that perform $2 \times N_n$ 8-bit integer multiplications in parallel. The products of these multiplications are summed and accumulated by an adder tree and a 32-bit accumulator, implementing the operations of $\sum_{i=1}^{N_n} b_i \times a_i$ and $\sum_{i=1}^{N_n} b_i \times d_i$ in parallel. Since the two 8×8-bit integer multiplications of $b_i \times a_i$ and $b_i \times d_i$ share the common multiplier, we can pack these $2 \times N_n$ multiplications into N_n DSP48E1 block. The procedure of the packed multiplication and the hardware structure of the packed multiplier are shown in Fig. 4b, c, respectively. Firstly, the 8-bit integer a_i is shifted and added with the other 8-bit integer d_i by utilizing the pre-adder in the DSP block, and the 25-bit result is then multiplied with the common 8-bit multiplier b_i using the 25×18-bit hard multiplier. Finally, a 17-bit calibration number is added to the product by utilizing the 48-bit accumulator to eliminate the error caused by the sign extension operation when $b_i \times d_i$ is negative [17]. Moreover, the 17-bit calibration number is calculated in advance by a simple logic circuit. In the approach of [18], up to eight products of the multiplications can be accumulated. However, it can only be applied to the DSP48E2 blocks due to the limitation on the width of the multiplier [19]. In contrast, our proposed scheme can be adopted for both DSP48E1 and DSP48E2 architectures. Compared to the approach of [4], our scheme consumes less logic resource since the operation of $a_i << 16 + d_i$ is conducted by the pre-adder inside the DSP48E1 block and only a small amount of logic resource is consumed to generate the calibration number. Moreover, the PE array is instanced for N_b times to further improve the throughput by introducing batch processing. In total, the PE arrays consume $N_r \times N_p \times N_n \times N_b$ DSP48E1 blocks to perform $2 \times N_r \times N_p \times N_n \times N_b$ 8-bit MAC operations in each clock cycle. The MAC results of size $N_r \times N_m \times N_b$ are generated in parallel, where N_m is the number of MAC results generated by each CU, and it equals to $2 \times N_p$.

Due to the data broadcasting structure and shared controlling logic, implementing a large PE array on FPGA devices can lead to high fan-out problems, which may reduce the fmax of the final design [20]. To avoid this problem, an exclusive loop controller is designed for each CU to reduce the fan-out of the controlling logic, and the data broadcasting is only performed within each CU. This multiple CU architecture has a significant advantage in scalability, which enables the proposed accelerator to be implemented on any type of FPGA device with near linear performance growth as the number of on-chip logic and DSP hardware resources increases.

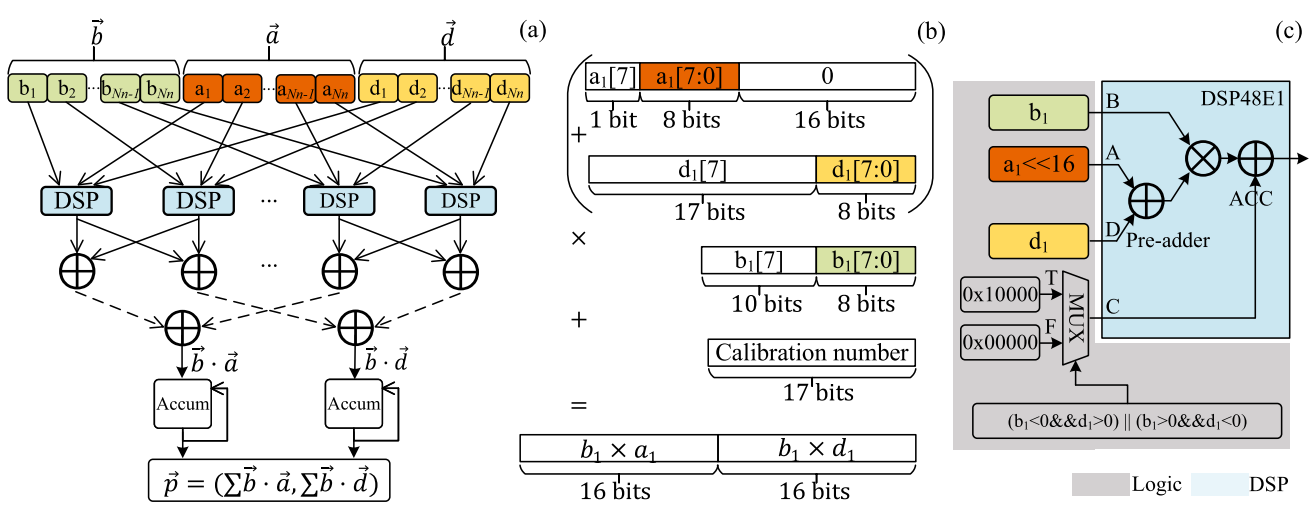

Fig. 4 DSP efficient PE design. **a** The architecture of a PE. **b** The procedure of the packed multiplication. **c** Mapping the packed multiplication into a DSP48E1

3.2 On-chip memory system design

The memory system of the accelerator is mainly composed of a group of on-chip buffers and external memory access units (namely the feature load, feature store unit, and weight load units). The on-chip buffer caches the serial input read from the external memory, reorders, and sends the data to the $N_r \times N_b$ CUs for parallel computation. The local data buffers can significantly reduce the data access latency of the PE array and improve the data reuse rate to relieve the pressure on external memory bandwidth. Moreover, a ping-pong buffer architecture is also proposed to overlap the data transfer with CU computation. The external memory access units are responsible for performing the high-throughput external memory access, where the burst memory access transfer is utilized.

As shown in Fig. 5, the feature buffer is constructed by $2N_{RAM}$ line buffers with one shared input port and N_r parallel output ports. The depth and the width of each line buffer equal to D_f and $N_m \times N_b$ bytes, respectively. The feature map data are fetched from external memory and stored in the line buffers in a zig-zag way, in which the address mapping sequence is controlled by the input multiplexer. One major benefit of this data caching scheme is that burst memory transfer can be utilized to maximize the external memory bandwidth since the physical address of the input data is continuous within each line. A total number of $2N_{RAM}$ buffer lines is designed to form a ping-pong structure, which overlaps the data transferring procedure with the convolution computation to improve the efficiency of the accelerator. Detailed pipelining strategy is explained in Sect. 3.4.

A total number of $N_r \times N_m \times N_b$ vector outputs are generated by the PE arrays in parallel in every L_{CU} cycles, which correlates with the convolution calculation mapping that is introduced in Sect. 3.3. However, the result must be written back to the external DDR SDRAM in a serial way. This imbalance raises two design challenges for the Feature Store unit, namely converting parallel data into serial data and maximizing external memory bandwidth utilization. A ping-pong buffer is designed to address the first issue. As shown in Fig. 6, once the parallel outputs are generated by the PE array, the ping-pong buffer starts to cache the result. In each cycle, $N_m \times N_b$ out of the $N_r \times N_m \times N_b$ outputs results are selected and stored in one of the ping-pong buffer. While converting the parallel convolution results of the current iteration, the converted serial results of previous iteration stored in another ping-pong buffer are being written to the external memory concurrently. Secondly, burst memory transfer is also utilized in the data store procedure to maximize the external memory bandwidth and minimize the latency. A total of N_r cycles are needed to store the parallel outputs which is generated every L_{CU} cycles. Therefore, to avoid any stall operations in the Feature Store unit or the PE array, the following condition must be satisfied: $L_{CU} > N_r$. In deep convolution neural network algorithms, the convolution operations are generally performed on vector data with large sizes, which can normally meet this requirement.

3.3 Convolution task mapping

Convolution calculation occupies the major workload in CNN inference computation [21]. As illustrated in Algorithm 1, it performs the inner product of the neural network weight W and the input feature map FI. The input, output feature maps and weights are of the size $N \times R \times C$, $M \times R' \times C'$ and $M \times N \times K \times K'$, respectively, and S denotes the stride of the convolution. Both parallelism mapping and data caching should be taken into concern together when mapping the convolution calculation on the proposed accelerator. The former optimization finds the best way that maps the convolution algorithm on the PE array that can maximize the acceleration of the convolution

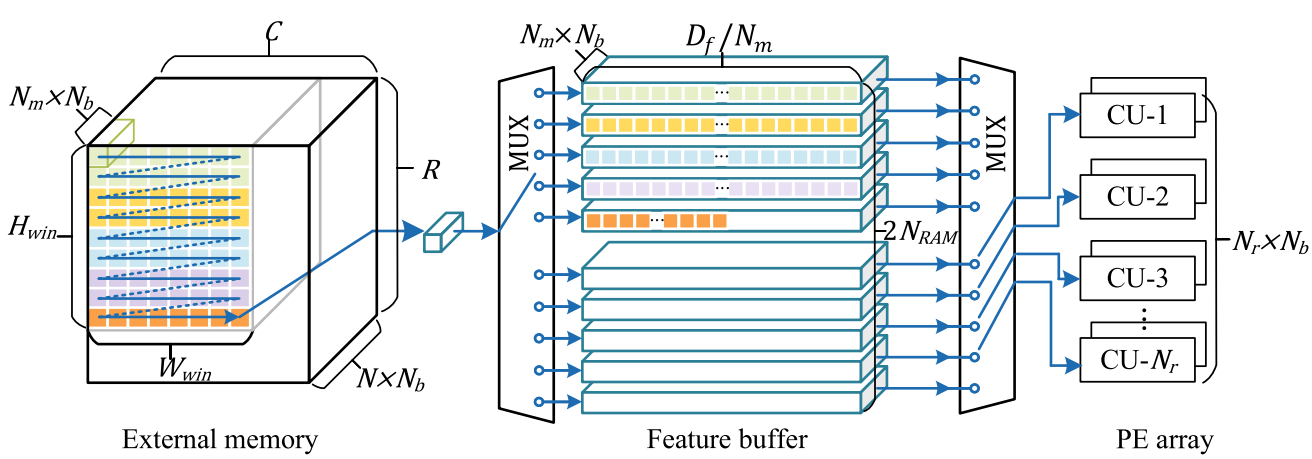

Fig. 5 Feature buffer design. The line buffers cache the serial data from the external memory, and output parallel data streams to the CUs

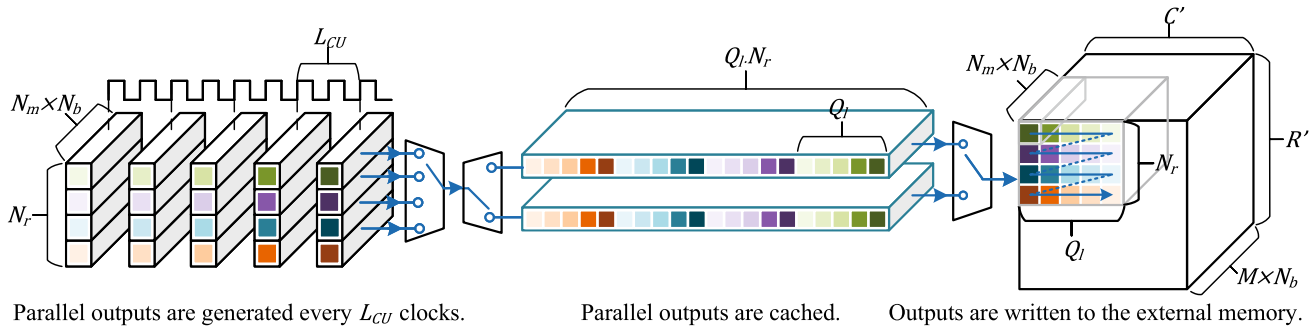

Fig. 6 Output buffer design. Every L_{CU} clocks, a total of $N_r \times N_m \times N_b$ outputs are generated. The output buffer caches them and writes them back to the external memory sequentially

computation by executing as many as parallel operations in each cycle. The latter optimization targets utilizing on-chip buffers to enhance on-chip data reuse rate to improve external memory efficiency.

Algorithm 1 Convolution dataflow

```
for rr = 1 to R' do
    for cc = 1 to C' do
        for mm = 1 to M do
            for kk = 1 to K do
                for kk' = 1 to K' do
                    for nn = 1 to N do
                        FO[rr][cc][mm] += FI[rr*S+kk][cc*S+kk'][nn] * W[kk][kk'][nn][mm]
                    end for
                end for
            end for
        end for
    end for
end for
```

Parallelism mapping From Algorithm 1, it can be observed that there is no computation dependency among each output channel dimension *mm* and each output row dimension *rr*, so the calculation for multiple output channels and output rows can be conducted in parallel. As shown in Fig. 7, three degrees of parallelism are exploited in the proposed accelerator, i.e., N_m and N_r convolution kernels are executed in parallel along the output channel

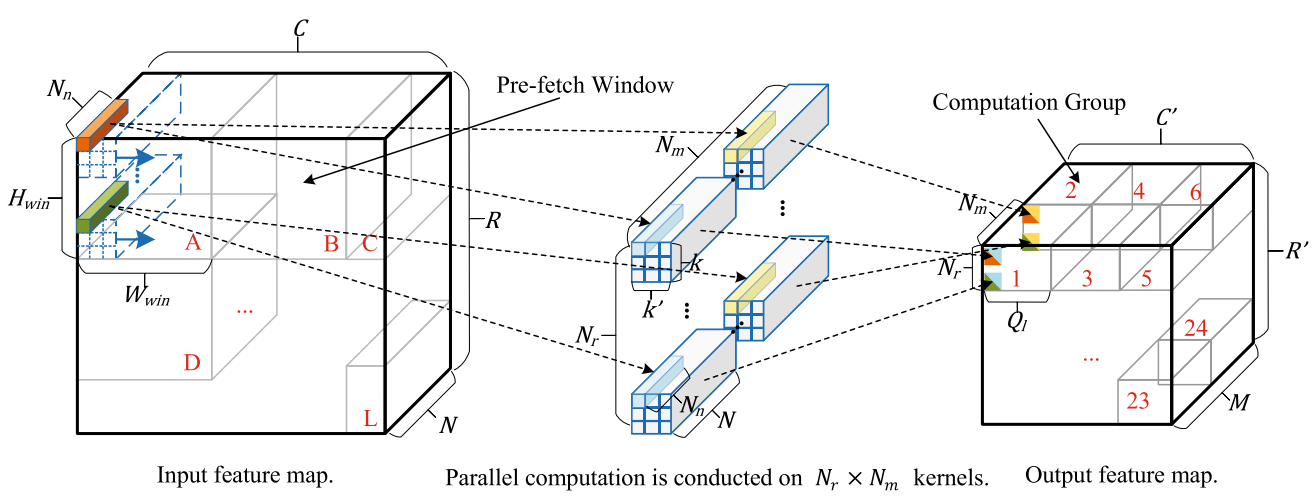

Fig. 7 Mapping the convolution computation into one PE array

and the input row dimensions, respectively, while N_n MAC operations are conducted concurrently within each convolution kernel to further accelerate the computation. For a convolution kernel of size $K \times K' \times N$, the MAC results of size $N_r \times N_m$ are generated every $L_{CU} = K \times K' \times \lceil N/N_n \rceil$ cycles. The workload of N_m convolution kernels along the output channel direction is carried by one CU, where every two convolution kernels are mapped to a PE since they share the same input feature map, which matches the multiplication packing scheme proposed in Sect. 3.1. During the convolution, as shown in Fig. 3, each CU gets its own input feature map data from the feature buffer, and those data are shared by the PEs within a CU. At the same time, weights are loaded and shared among the N_r CUs, and each PE gets $2 \times N_n$ pixels of weights correlating with two convolution kernels. Therefore, a total of $N_r \times N_m \times N_n$ MAC operations are conducted concurrently by one PE array. Moreover, the calculation of N_b different inputs are also conducted concurrently to further improve the parallelism and the weights reuse.

Data caching To produce one output feature pixel, the accelerator needs to hold the amount of $K \times K' \times N$ pixels of feature map and weight data for parallel computation, respectively. As the convolution kernel slides over the whole input feature map, more feature data have to be fetched repeatedly from external memory [16], while the weight data can be reused. To fit the depth of the on-chip feature buffer, we propose to divide and execute each convolution layer as multiple computation groups, and each group consists of $N_r \times Q_l \times N_m$ times of convolution computation defined in Algorithm 1. As shown in Fig. 7, the depth and the height of each computation group (i.e., N_m and N_r) fit the size of the PE array, while the width Q_l is determined by the depth of the feature buffer. Consequently, the number of computation groups along the row, column, and channel dimensions can be calculated by

$$\begin{aligned} G_l^r &= \lceil R_l'/N_r \rceil, \\ G_l^c &= \lceil C_l'/Q_l \rceil, \\ G_l^m &= \lceil M_l/N_m \rceil. \end{aligned} \tag{12}$$

The whole range of the input feature data that corresponds to one computation group is defined as a pre-fetch window. As shown in Fig. 7, the height and width of the pre-fetch window can be calculated by

$$\begin{aligned} H_{win} &= (N_r - 1) \times S + K, \\ W_{win} &= (Q_l - 1) \times S + K'. \end{aligned} \tag{13}$$

Consequently, the whole input feature map is divided into $G_l^r \times G_l^c$ pre-fetch windows during execution on the accelerator, and the data within a pre-fetch window are shared by G_l^m convolution groups along the output channel dimension. For instance, computation group 1 and group 2 share the pre-fetch Window A.

Since there is a fixed interval of S lines between the feature maps required by the adjacent convolution tasks, each line buffer in the feature buffer is designed to store at most S lines of feature maps to avoid possible data access collision. So, a total number of $N_{RAM} = \lceil H_{win}/S \rceil$ line buffers are configured in the feature buffer. Given the depth of the line buffer as D_f, the width of the pre-fetch window equals to

$$W_{win} = D_f/(N \times S). \tag{14}$$

Considering the correlation between the Q_l and W_{win} which is shown in Eq. (13), the width of the computation group is restricted by

$$Q_l = \lfloor (W_{win} - K_l)/S_l \rfloor + 1. \tag{15}$$

For instance, in Fig. 5, assuming that the size of the convolution kernel is of the size $3 \times 3 \times 1$ and $S = 2$, $N_r = 4$, then the height of the perfetch window equals to $H_{win} = (N_r - 1) \times S + K = (4 - 1) \times 2 + 3 = 9$, and the number of line buffers is $Ceil(H_{win}/S) = Ceil(9/2) = 5$.

The execution order of the computation groups has a significant impact on the on-chip data reuse rate and external memory bandwidth utilization. Two scheduling schemes are considered in this work. As shown in Fig. 7, the first task scheduling scheme (Scheme-A) launches the computation groups in the order of Group 1, 2,..., 24, which can maximize the reuse rate of cached feature map data since adjacent groups along output dimension M (e.g., Group 1 and 2) share the same pre-fetched window. The second strategy (Scheme-B) is to execute the computation groups in the order of Group 1, 3, 5,..., 24, which can improve the weight data reuse rate, since these groups share the same weights. The efficiency of these two schemes are determined by multiple factors, such as the size of each group, the neural network structure, and the external memory bandwidth limitation. Therefore, experiments have to be conducted on the target FPGA platform to measure the execution time and bandwidth utilization before making the final decision. In Sect. 4.4, we will provide detailed experiment results and compare the pros and cons of these two scheduling schemes based on the Roofline model. Algorithm 2 summarizes the optimized convolution dataflow. In this dataflow, each group of computation tasks is performed sequentially, and its corresponding data are also cached sequentially. Moreover, a total number of G_l^m groups share the same input feature map. As shown in the Line 11 and Line 17 of Algorithm 2, a total of $N_m \times N_r \times N_n$ MAC operations are conducted concurrently.

Algorithm 2 Throughput optimized convolution dataflow

```
_buffer_ FT_Buf[H_win][W_win][N]
_buffer_ WT_Buf[K][K'][N][N_m]
_buffer_ psum[N_r][N_m]
for g_r = 1 to G_l^r do
    for g_c = 1 to G_l^c do
        Cache the feature map FT_Buf[H_win][W_win][N]
        for g_m = 1 to G_l^m do
            Cache the weights WT_Buf[K][K'][N][N_m]

            for cc = 1 to Q_l do
                #pragma unroll by a factor of N_m × N_r
                for [i = 1 to N_r] × [j = 1 to N_m] do
                    psum[i][j] = 0
                    for kk=1 to K do
                        for kk' = 1 to K' do
                            for pp = 1 to N/N_n do
                                #pragma unroll by a factor of N_n
                                for nn = 1 to N_n do
                                    psum[i][j] += FT_Buf[i × S + kk][cc × S + kk'][pp × N_n +
    nn] · WT_Buf[kk][kk'][pp × N_n + nn][j]
                                end for
                            end for
                        end for
                    end for
                    FO[g_r × N_r + i][g_c × gp_c + cc][g_m × N_m + j] = LeakyReLU(psum[i][j])
                end for
            end for

        end for
    end for
end for
```

3.4 Task scheduler design

Task scheduler is designed to synchronize the state of each function unit and dynamically schedule the convolutional computation, which reduces the control workload of the processor. As illustrated in Fig. 7, the convolution operation is divided into multiple computation groups, and the execution of each group is dynamically scheduled by the task scheduler unit. Besides managing the execution order of the computation groups, the task scheduler unit also synchronizes the state of each function unit. As shown in Fig. 8, at the beginning of each layer, the feature and weight data are first loaded from the external memory and cached in the on-chip buffers. When finishing loading the required data by the first computation group, (i.e., Time 1), the task scheduler unit sends signals to start the PE array. While the PE array conducting the convolution, the Feature Load unit and Weight Load unit continue loading feature map and weight data for the subsequent groups, i.e., from time 1 to 2. Since the convolution group 2 and group 1 share the same input feature map, computation group 2 can be started upon finishing group 1 and loading the required

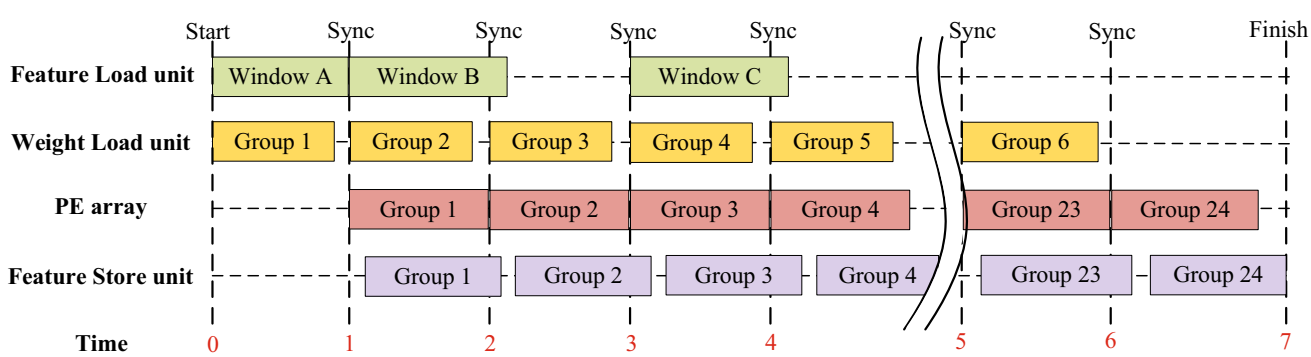

Fig. 8 Task-level pipeline strategy

weights data, i.e., the time 2. Additional attention should be paid that, at this time point, the loading of pre-fetch window B does not need to be finished, since window A is still in use.

In the proposed architecture, synchronization is only performed when new input feature map or weight data were required, and each functional unit runs independently for the rest of the time, which reduces the risk of accelerator stall caused by high-latency off-chip memory access. In addition, a larger feature map pre-fetch window will reduce the frequency of synchronization, which helps to improve the efficiency of the accelerator, but will consume more on-chip memory resources. Therefore, the size of the perfetch window has to be selected carefully to balance the performance and hardware cost, and it will be introduced in the design exploration Sect. 4.3.

3.5 Post-processing

The detailed processing procedures of the post-processing algorithms in YOLO detector is illustrated in Fig. 9. The YOLO algorithms predict the coordinates of bounding boxes relative to the location of the grid cells, where a sigmoid function constrains the center coordinates of the bounding box to fall in the grid cell [12]. The exponent function extracts the height and width of the bounding box, which ensures the width and height are greater than 0. The confidence scores are extracted from the network's prediction by multiplying the outputs of the softmax function and the probability of containing an object. Then, the confidence scores under the threshold are filtered to reduce the amount of candidate boxes. Once finished the coordinates and confidence scores extraction, the NMS is carried out to further filter the redundant predictions. In the NMS stage, the confidence scores are sorted firstly, and the box corresponding to the largest score is selected as one candidate box for this class. Then the IOU values between this box and all other boxes are calculated and those boxes whose IOU values are above the predefined threshold are discarded. This procedure continues until all boxes are either selected or discarded [10].

The coordinates and confidence scores extraction are computational-intensive tasks consisting of a large amount of sigmoid, exponent and softmax functions. These nonlinear functions require complicated floating point operations. In contrast, the NMS algorithm is generally control-intensive since it contains many indexing, sorting and grouping operations. In most accelerator designs [4–6, 9], those functions are implemented on general-purpose processors, such as the ARM or x86 CPUs, as software programs. Since these CPUs are mainly hard-core processors with high computing performance, the implemented programs can be executed with very low latency delivering real-time detection results. However on FPGA fabric-only devices, only softcore CPU processors, such as the MicroBlaze [22] and NIOS [23] processors, are available with very limited computational capability. Therefore, the above algorithms in the post-processing stage of YOLO detectors have to be implemented in dedicated hardware circuits to achieve real-time processing speed.

Coordinates calculation The location of a prediction box is represented by the coordinates of two diagonal points, i.e., (x_1, y_1) and (x_2, y_2). According to the YOLO algorithm [12], the normalized location x_1 can be calculated by

$$x_1 = [(\sigma(t_x) + c_x) - (0.5 \cdot p_w \cdot e^{t_w})] \cdot scale, \tag{16}$$

where σ denotes the sigmoid function, and t_x and t_w are coordinates information extracted from the CNN outputs, and c_x is the offset from the top left corner of the image, and p_w represents the width of the bounding box prior. The architecture of the proposed hardware location decoding unit is shown in Fig. 10. Since t_x, t_w, c_x and p_w are all quantized in 8-bit fixed-point format, the sigmoid and exponential functions can be implemented by look-up table with low hardware costs. In the location decoding unit, two look-up tables with 256 entries are utilized to

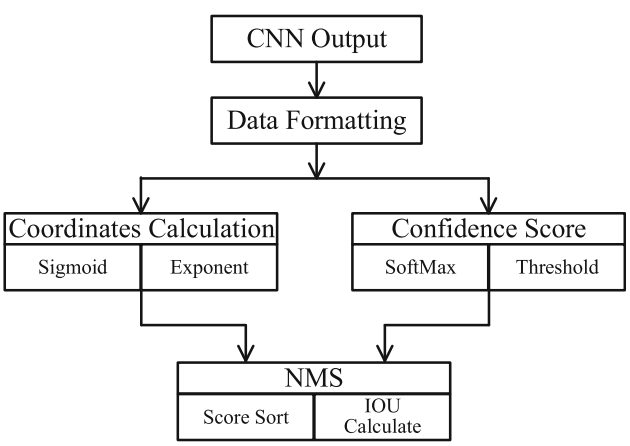

Fig. 9 Work flow of the post-processing

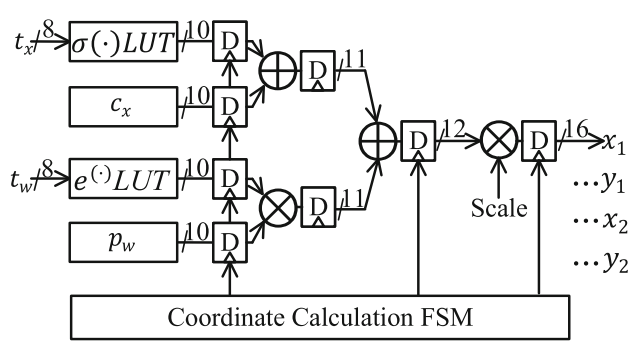

Fig. 10 Location decoding unit

generate the outputs of the sigmoid and exponential functions. Moreover, the intermediate data and subsequent operations are quantized to further reduce the hardware resource consumption and calculation latency. The output coordinates are quantized to four 16-bit fixed-point numbers, since the resolution of the input picture equals to 416×416, and the accuracy of the 16-bit numbers is enough. The whole procedure of the coordinate calculation is fully pipelined and controlled by the finite-state machine (FSM) to improve the hardware efficiency.

Confidence scores calculation The confidence scores equal to the product of the softmax function and the probability of containing an object. Its architecture is shown in Fig. 11, which can be divided into three stages. In the first stage, the maximum value is extracted from the serial inputs. In the second stage, the subtraction is conducted between the inputs and the maximum value, and their exponentiation and summation operations are calculated. In the last stage, the floating point multiplication and division are carried out. It should be taken into account that during the confidence score calculation, only the multiplication and division in the third stage are carried out using the float data.

$$Si = \frac{e^{(x_i - X_{\max})}}{\sum_{j=1}^{N} e^{(x_j - X_{\max})}}, \tag{17}$$

NMS unit After the coordinates and confidence scores were calculated, a large number of redundant candidate boxes for the same target are generated, and these candidate boxes may overlap with the neighboring boxes. In order to solve this problem, the NMS is carried out to find the best target bounding box and eliminate the redundant boundary boxes. A hardware-friendly NMS algorithm was proposed by [10], where the sorting of confidence scores was omitted. The design achieved a very low latency at the cost of using 430 on-chip DSP blocks. This hinders the implementation of the hardware NMS on some resource-constrained devices. Inspired by that work, a resource-efficient NMS algorithm is proposed in this work, which is shown in Algorithm 3. For each input bounding box, its IOU between each selected box is calculated. The selected box will be replaced when the IOU is beyond the threshold, and the confidence score is less than that of the input box. The input box is inserted to the selected boxes when the IOU between the input box and each of the selected boxes is less than threshold. The hardware architecture of the NMS unit and the overall post-processing unit is shown in Fig. 12. The proposed NMS hardware unit is mainly composed by a selected box buffer and an IOU calculation unit. The NMS unit connects with the coordinate and score calculation units using two channels, which forms a deep pipeline structure to reduce the off-chip data access. A threshold unit is designed to filter out the boxes with low confidence score, which relieves the workload of the NMS unit.

Algorithm 3 NMS implementation

```
Input: BOX_in, IOU_thr
Output: BOX_sel[CLS][50]
for each b_in in BOX_in do

    INSERT = True
    REPLACE = True
    for each b_sel in BOX_sel[b_in.cls] do
        if IOU(b_sel, b_in) > IOU_thr then
            INSERT = False
            if b_in.score > b_sel.score then
                if REPLACE == True then
                    b_sel = b_in
                    REPLACE = False
                else
                    b_sel = NULL
                end if
            end if
        end if
    end for

    if INSERT == True then
        insert(BOX_sel[b_in.cls], b_in)
    end if
end for
```

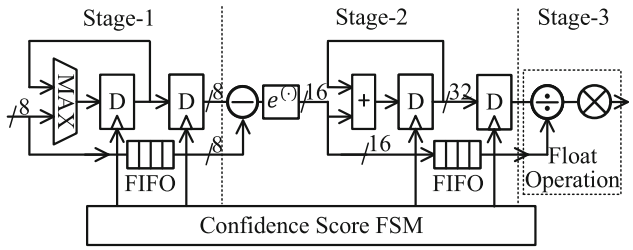

Fig. 11 Softmax unit

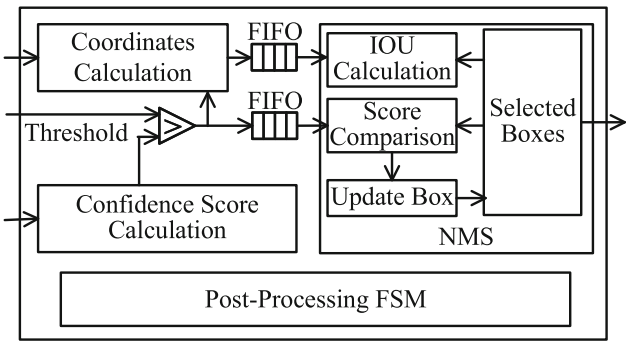

Fig. 12 Hardware architecture of the post-process unit

4 Performance modeling and design space exploration

Design space exploration plays an important role in FPGA-based accelerator design, which enables the system to achieve a balance between performance and resource consumption. In this section, the performance and resource consumption models are first built, and comprehensive design space exploration is then carried out to find the optimal parameters for the PE array and the on-chip memory system. Moreover, the performance of the two task scheduling schemes discussed in Sect. 3.3 is also measured and compared by using the Roofline model.

4.1 Accelerator performance

According to the Roofline model [24, 25], the throughput is constrained by either the bandwidth roof (R_{bw}) or computational roof (R_{cp}). The computational roof measures the peak throughput provided by the computation resource in the system. The bandwidth roof measures the throughput that the memory system can support [25]. The overall throughput equals to $\min(R_{bw}, R_{cp})$. The proposed design has a total of $N_r \times N_b$ CUs. Each CU performs $N_m \times N_n$ MAC operations concurrently. Considering a MAC operation as two operations, i.e., addition and multiplication, the computational roof equals to

$$R_{cp} = 2 \times N_n \times N_r \times N_m \times N_b \times Freq. \tag{18}$$

Once finishing loading a feature pre-fetch window, the feature will be used by G_l^m computation groups along the output channel dimension. Therefore, the total pixels of feature map data loaded from and written to the external memory for the l-th layer of convolution computation is

$$H_l^f = (G_l^c \times G_l^r \times W_{win} \times H_{win} \times N + C' \times R' \times M) \times N_b. \tag{19}$$

For each computation group, an amount of $K \cdot K' \cdot N \cdot N_m$ weights is required, so the total amount of weights that needs to be loaded from external memory during one layer of calculation equals to

$$H_l^w = G_l^c \times G_l^r \times K \times K' \times N \times M. \tag{20}$$

The arithmetic intensity, which is calculated as the ratio of the amount of calculation to the amount of external memory access, is denoted as

$$I = \frac{\#OP_l}{H_l^f + H_l^w} \tag{21}$$

where the $\#OP_l$ denotes the number of operations performed in the l-th convolution/FC layer, and it equals

$$\#OP_l = 2 \times K \times K' \times N \times C' \times R' \times M \times N_b. \tag{22}$$

Further, the peak throughput supported by the memory system equals to the product of the arithmetic intensity and external memory bandwidth, i.e.,

$$R_{bw} = I \times BW_{DDR}, \tag{23}$$

where BW_{DDR} denotes the bandwidth of the external memory. The theoretical execution latency for the l-th convolution/FC layer equals to the quotient of the number of MAC operations and the throughput, i.e.,

$$T_l = \frac{\#OP_l \cdot \lceil R'/N_r \rceil \cdot N_r/R'}{min(R_{cp}, R_{bw})}. \tag{24}$$

Equation (25) measures the latency of performing the deep CNN algorithm, which equals to the sum of latency of all the layers, while the overall throughput equals to the quotient of the total amount of calculation and the T_{sum}.

$$T_{sum} = \sum_{l=0}^{L} T_l \tag{25}$$

4.2 Hardware resource consumption

The proposed accelerator can be configured by a group of hardware parameters, including the N_n, N_r, N_m, and N_b related to the parallelism and the D_f related to the depth of

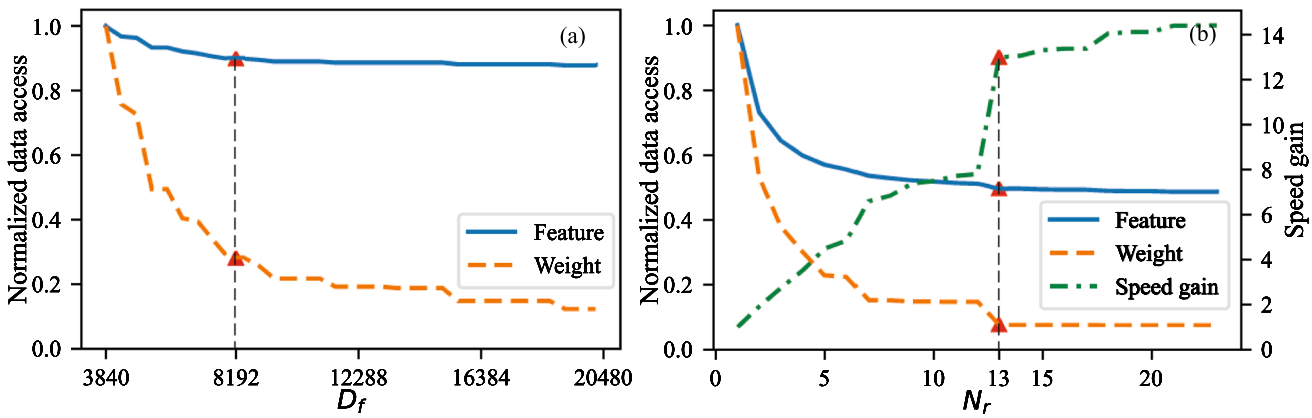

Fig. 13 Selection of D_f and N_r. **a** The correlation between external memory access and D_f. **b** The external memory access and speed gain for different N_r

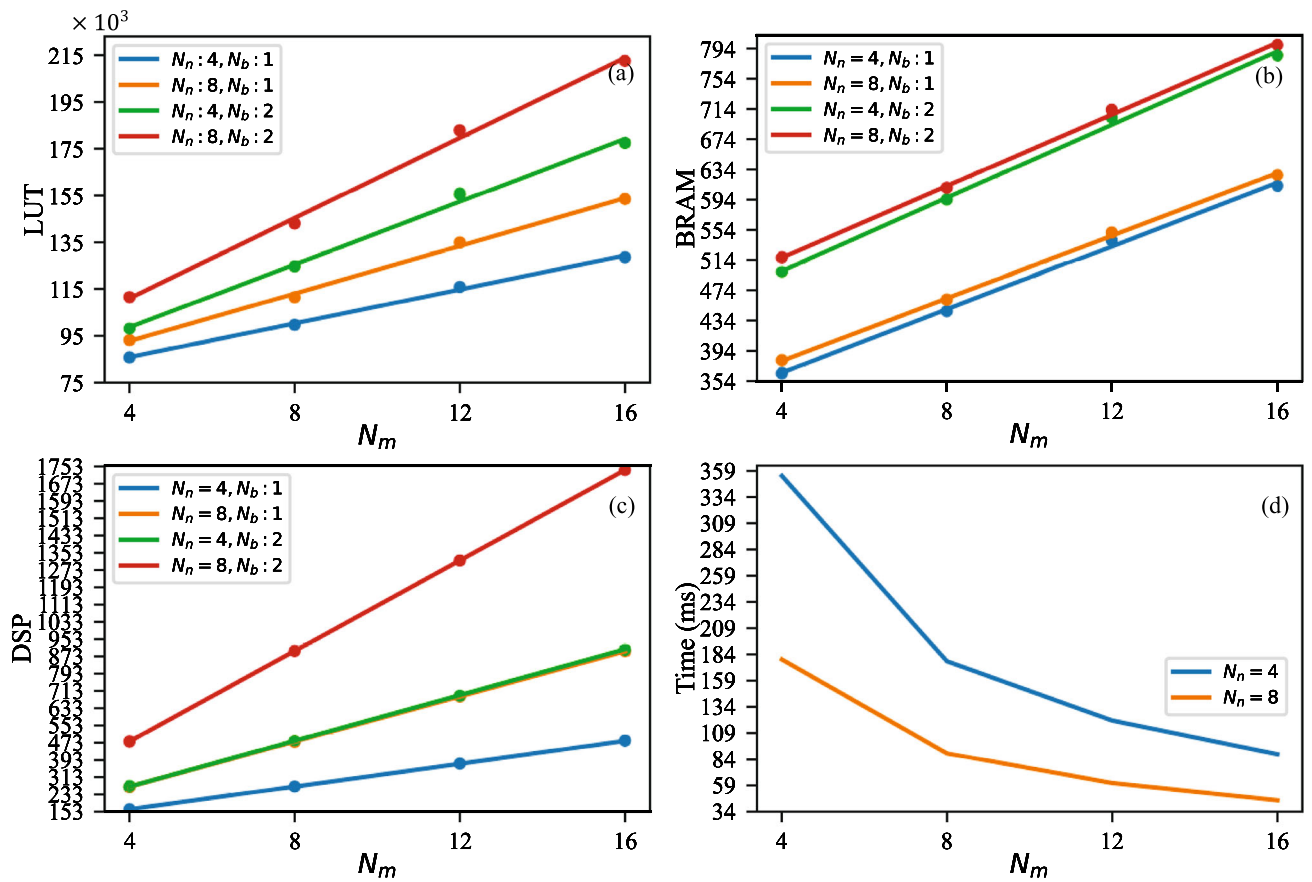

Fig. 14 Resource consumption estimation. The target device is Xilinx V7-690T and the other parameters are $N_r = 13$ $D_f = 8192$. **a** Logic resource. **b** Block RAMs. **c** DSP blocks. **d** Inference latency

the feature buffer. Hardware resource consumption is positively correlated with these parameters. In order to quantitatively guide the accelerator design, the logic, memory, and DSP consumption models correlated with these parameters are defined. The PE array accounts for the most of DSP consumption which performs $N_n \times N_r \times N_m \times N_b$ multiplications in every clock cycle concurrently. Two 8-bit multiplications are done by one DSP block, so the DSP consumption is modeled by

$$C_{dsp} = \frac{1}{2} \times N_n \times N_r \times N_m \times N_b + C_1, \tag{26}$$

where C_1 denotes the DSP resource consumed by the rest of parts of the accelerator, such as the address generator.

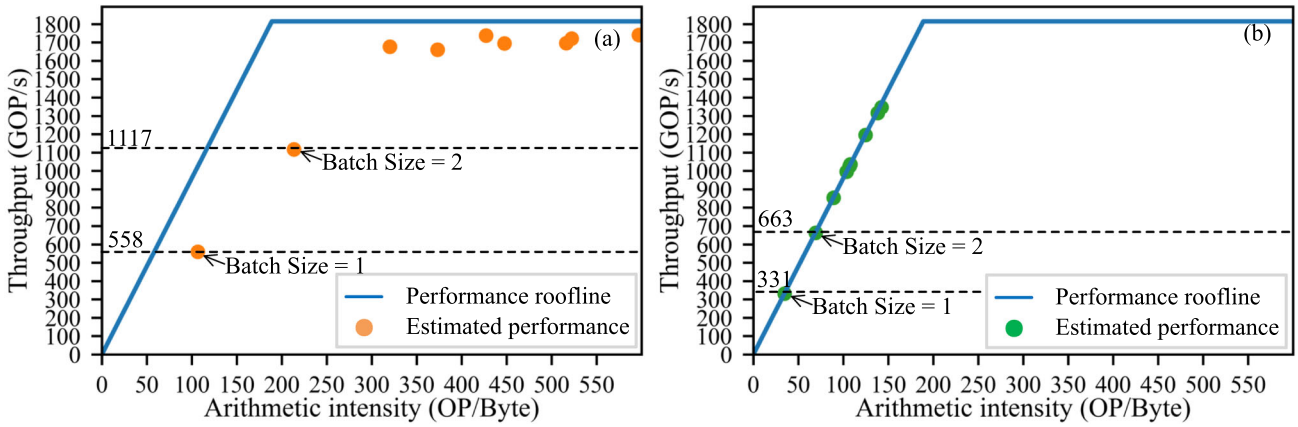

Fig. 15 Roofline models for the two schemes, where the estimated throughput correlates with the batch size from 1 to 8 is shown. **a** Scheme-A. **b** Scheme-B

The feature buffer and weight buffer consume the most of on-chip RAM resources. The feature buffer is composed by $N_r + \lceil (K/S) \rceil - 1$ dual-ports RAMs with the depth of $D_f \times S \times N$. When modeling the consumption of on-chip RAMs, not only the size of the buffer needs to be considered, but also the width of the port. A few more RAM blocks are needed to achieve a large port width, for the on-chip memory block has its limitation of maximum port width. In this work, the width of the feature buffer equals to $8 \times N_m \times N_b$ bit, and the width of the weight buffer equals to $8 \times N_n \times N_m$ bit. The total on-chip memory consumption is estimated by (27)

$$C_{mem} = C_2 \times N_n \times N_m + C_3 \times N_r \times N_m \times D_f \times N_b + C_4, \tag{27}$$

where C_2, C_3, C_4 denotes the factor the parameters that measure the number of BRAM consumed by the weight buffer, feature buffer, and the rest of the modules.

The PE array takes up most of the logic consumption to generate the adder tree and the accumulator. The logic consumption for each adder tree correlates with the number of additional operations, i.e., N_n, and the logic consumption for a 32-bit accumulator can be seen as a constant. Moreover, there are a total of $N_r \times N_m \times N_b$ adder trees and accumulators in the PE array. So, the logic consumption is estimated by

$$C_{logic} = (C_5 \times N_n + C_6 + C_7) \times N_r \times N_m \times N_b + C_8, \tag{28}$$

where $C_5 \times N_n + C_6$ and C_7 measure the logic consumed by an adder tree and a 32-bit accumulator, respectively, and C_8 denotes the logic consumed by other functional units.

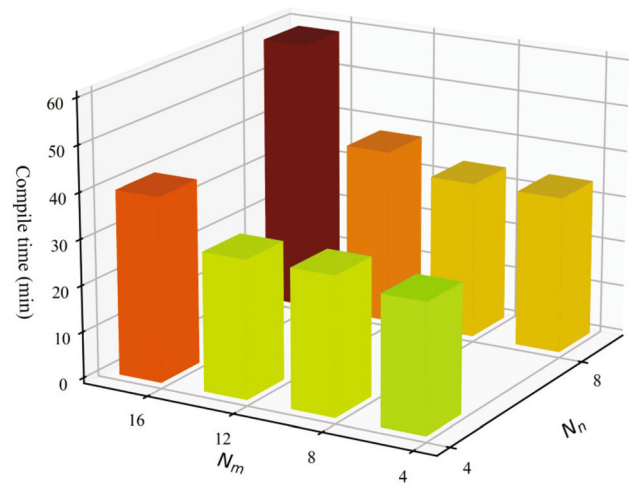

Fig. 16 Compilation time for different hardware parameter setting ($N_r = 13, N_b = 1$)

4.3 Design space exploration

The final goal of the design space exploration is to find the highest throughput while keeping the three types of hardware resource consumption within the device limitation. In the design space exploration, the hardware resource consumption is constrained under 70% to prevent timing issues, i.e.,

$$\begin{aligned} & max(Throughput) \\ & s.t.: \; C_{dsp} \leq 0.7 \times DSP_{lim} \\ & \qquad C_{mem} \leq 0.7 \times MEM_{lim} \\ & \qquad C_{logic} \leq 0.7 \times LOGIC_{lim}, \end{aligned} \tag{29}$$

where DSP_{lim}, MEM_{lim} and $LOGIC_{lim}$ denotes the available DSP, on-chip memory and logic resource, respectively. According to (24), both the computation roof and

bandwidth roof should be considered during the exploration of the peak performance. Moreover, the power consumption of the system and the frequency of external memory access are usually positively correlated with each other. Therefore, the external memory access needs to be limited to a relatively low level to reduce the system power consumption and the external memory bandwidth pressure. According to (19) and (20), the feature and weight access are correlated with the number and size of the convolution group which depend on N_r and D_f. As shown in Fig. 13a, the external memory access drop sharply with the increase in D_f, when D_f is less than 8192. However, the decline of external memory access tends to slow down when D_f is greater than 8192, and the on-chip memory consumption correlates with D_f. Taking the on-chip memory consumption and the external memory access into consideration, the D_f is set to 8192. Similarly, as shown in Fig. 13b, the decline of external memory access slows down when N_r is greater than 13. Moreover, the speed-up gain reaches a saturation value when $N_r > 13$, which is caused by the low PE array utilization when conducting the last few layers of convolution, where the size of the feature map in dimension R' is relatively small. According to the external memory access and the speed-up gain, N_r is set to 13 to achieve the most efficient speed-up gain and low external memory access.

Subsequently, a few rounds of compilation are conducted to gather the resource consumption information correlated with different settings of parameters N_n, N_m and N_b, including the logic, on-chip memory, and DSP consumption. The inference latency and the resource consumption are shown in Fig. 14, and the parameters that achieve minimum latency under the device resource limitation are expected. Furthermore, routing or timing failures may be caused by high resource occupancy, so in this design, we constrain the consumption under 70% of the total hardware resource. In the final implementation, $N_n = 8$, $N_m = 16$, $N_r = 13$ and $N_b = 2$ are selected, and the achieved highest frequency is 160 MHz.

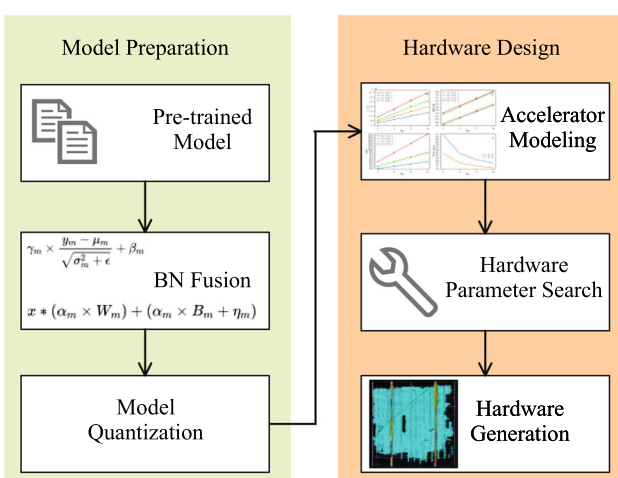

Fig. 17 Work flow of the accelerator design

4.4 Comparison of task scheduling methods

As discussed in Sect. 3.4, the scheduling Scheme-A maximizes the reuse of feature map, while the Scheme-B maximizes the reuse of weights. In this section, these two schemes are evaluated by using the Roofline model. The performance of Scheme-A has already been modeled in Sect. 4.1, so this section only focuses on modeling the performance of Scheme-B. In Scheme-B, the input feature map data are not shared among different groups, and a total number of $H_{win} \times W_{win} \times N$ input pixels are needed to be loaded from external memory before conducting one group of the convolutional task. Therefore, for the l-th convolution layer, the total amount of feature map data loaded from and written to the external memory in Scheme-B equals to

$$h_l^f = (G_l^c \times G_l^r \times G_l^m \times W_{win} \times H_{win} \times N + C' \times R' \times M) \times N_b. \tag{30}$$

For the weights, every $G_l^c \times G_l^r$ groups share the same data, so the amount of weights to be loaded for the l-th layer is

$$h_l^w = K \times K' \times N \times M. \tag{31}$$

Then, theoretical throughput can be calculated by using Eqs. (30), (31) and the performance models defined in Sect. 4.1. The final obtained Roofline models for both Scheme-A and Scheme-B are shown in Fig. 15, where the overall throughput when implementing YOLOv2 algorithm with different batch sizes are proposed. Special attention should be paid to when modeling the computational roof, and a total of 30% hardware resource is reserved. As shown in Fig. 15a, the arithmetic intensity increases linearly with the increase in batch size from 1 to 4, which is caused by the increase in off-chip bandwidth utilization. The increase in the arithmetic intensity slows down when the batch size is greater than 4, for the full utilization of off-chip bandwidth. As shown in Fig. 15b, the throughput is bandwidth-bounded. From the above comparison, it is clear that Scheme-A is superior to Scheme-B, and is thus adopted in the final implementation.

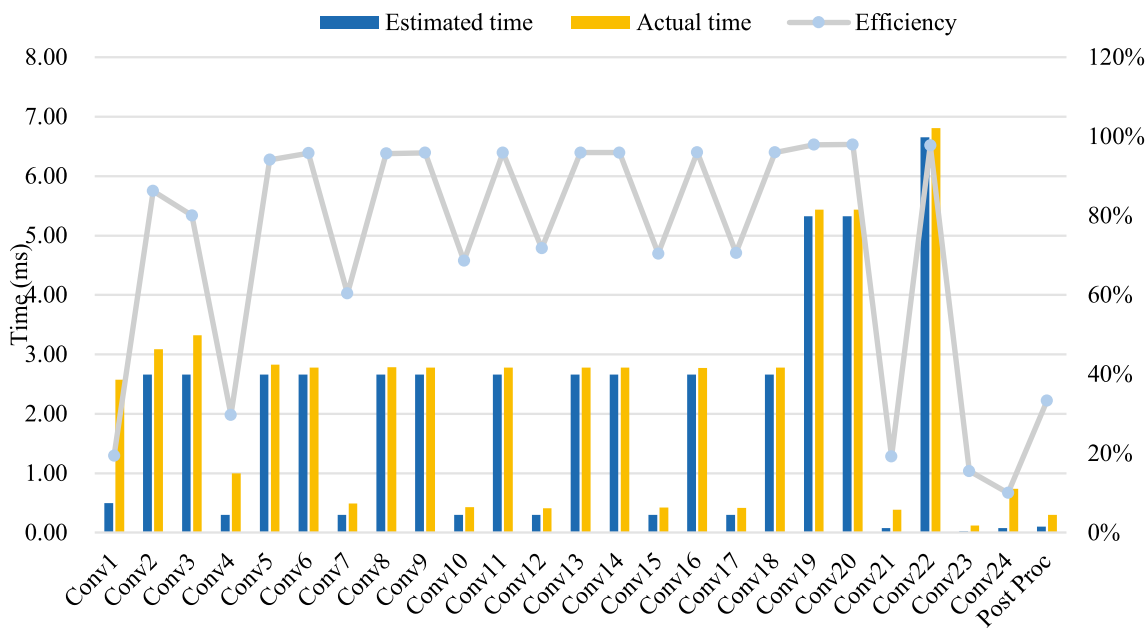

Fig. 18 Theoretical and actual execution time of each layer of the YOLOv2 algorithm on the VOC data set ($N_n = 8, N_m = 16, N_r = 13, N_b = 1$, Frequency = 180 MHz)

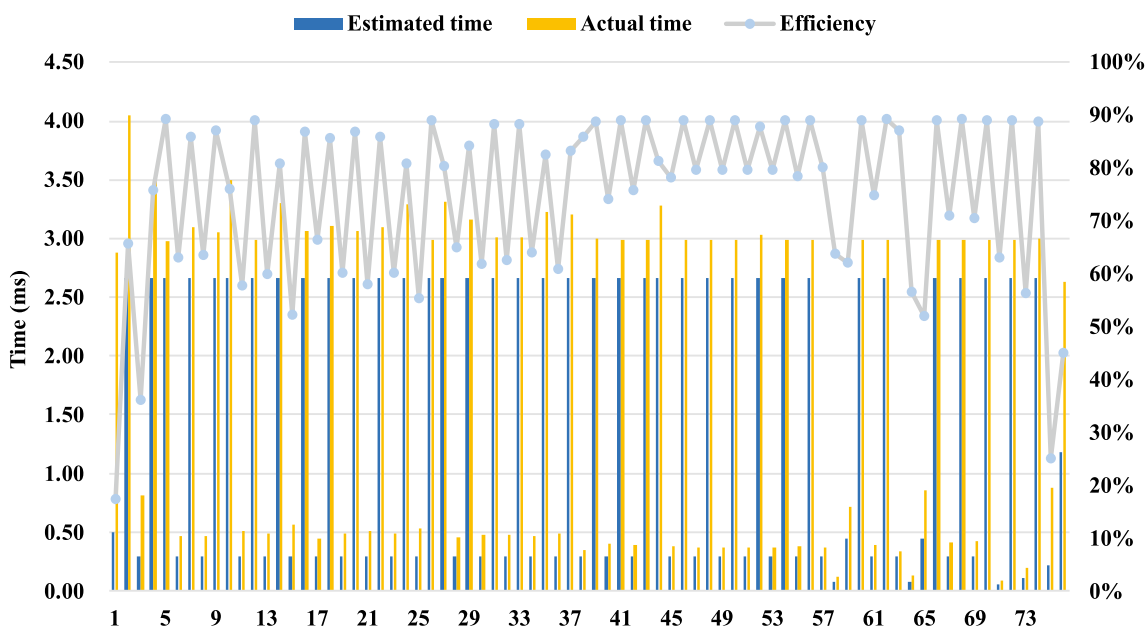

Fig. 19 Theoretical and actual execution time of each layer of the YOLOv3 algorithm on the VOC data set ($N_n = 8, N_m = 16, N_r = 13$)

5 System implementation and performance evaluation

5.1 Experimental setup

To evaluate the proposed accelerator, we utilized the KC705 and VC709 development boards, covering FPGAs with varying capacities. The KC705 board contains a low-end Xilinx XC7K325T FPGA with 203k LUTs, 840 DSPs, and 16Mbit BRAM. One DDR3 memory with a capacity of 1GB is connected to the FPGA device on the board with a transfer bandwidth of 64$^{@}$1600MTs. The VC709 development board contains a Xilinx XC7VX690T FPGA with 433k LUTs, 3600 DSPs, and 52.9Mbit BRAM. Two DDR3 memories, each with a capacity of 4GB and a transfer bandwidth of 64$^{@}$1600MTs, are connected to the FPGA device on the board. To compile the HLS and generate RTL, we utilized Vitis HLS 2021.2, and the bitstream was

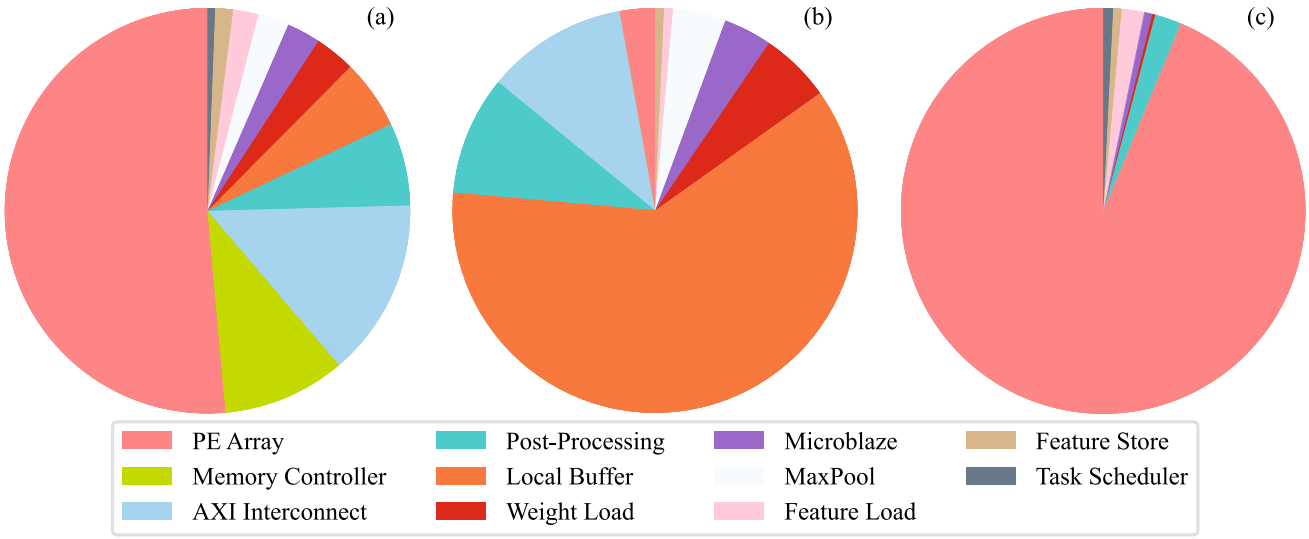

Fig. 20 Breakdown hardware resource of the accelerator with $N_n = 8$, $N_m = 8$, $N_r = 13$, and $N_b = 1$. **a** LUT consumption, **b** BRAM consumption, **c** DSP consumption

Table 2 Performance and resource consumption of the post-processing unit

	LUTs	DSP	BRAM (18k)	Latency (845 boxes)
Coordinates decode	3173	16	5	–
Score calculation	1343	0	2	–
NMS unit	1773	3	2	–
Misc unit	2604	0	11.5	–
Total	8893	19	20.5	0.38 ms

generated using Vivado 2021.2. The program for the host processor was compiled using Vitis 2021.2. The neural networks were trained using the MMDetection 2.14 framework [26] on the PASCAL VOC 2007 and VOC 2012 datasets [27]. We quantized the neural network using the scheme of BRECQ [13], optimizing the bit width of activations and network parameters to 8-bit. The platform used for the model quantization was composed of an Intel i7-6700k CPU, four 2080Ti GPUs, and 64 GB RAM. Approximately, 2.5 and 4 h were consumed in quantizing YOLOv2 and YOLOv3, respectively. The HLS compiling

Table 3 Comparison with state-of-the-art YOLOv2 and YOLOv2-Tiny implementations on FPGA

	ISCAS2021 [4]	NCAA2022 [9]	Ours	Ours	Ours	Ours	Ours
Benchmark model	YOLOv2-Tiny	YOLOv2	YOLOv2-Tiny	YOLOv2	YOLOv2-Tiny	YOLOv2	YOLOv2
Dataset	KITTI	VOC	VOC	VOC	VOC	VOC	VOC
MAP	77.1	75	53.5	76.7	53.5	76.7	76.7
Quantization (bits)	8	8	8	8	8	8	8
FPGA device	XC7Z045	ZU7EV	XC7K325T	XC7K325T	XC7VX690T	XC7VX690T	XC7VX690T
Technology (nm)	28	16	28	28	28	28	28
LUTs (K)	84/218	91.17/230	134.9/203	134.9/203	153.4/433	153.4/433	212.4/433
DSP Blocks	610/900	387/1728	687/840	687/840	897/3600	897/3600	1737/3600
Block RAM 18K	512/1090	461/624	551/890	551/890	627/2940	627/2940	799/2940
Frequency (MHz)	200	200	200	200	180	180	160
Batch size	1	1	1	1	1	1	2
Throughput (GOP/S)	464.5	100.3	406	418	492	525	914
On-chip power (W)	–	4.8	7.8	7.8	8.3	8.3	12.5
Total power (W)	10.2	–	12.3	12.3	19.8	19.8	24.1
Efficiency (GOPs/W)	–/45.5	20.9/–	52.1/33.0	53.6/33.9	58.8/24.8	62.7/26.5	72.7/37.9

Table 4 Comparison with state-of-the-art YOLOv3-related implementations on FPGA

	VLSI2022 [28]	ISCAS2020 [6]	Ours	Ours	Ours	Ours	Ours
Benchmark model	YOLOv3-Tiny	YOLOv3-Tiny	YOLOv3-Tiny	YOLOv3	YOLOv3-Tiny	YOLOv3	YOLOv3
Dataset	ILSVRC2015	–	VOC	VOC	VOC	VOC	VOC
MAP	73.1	–	58.1	75.4	58.1	75.4	75.4
Quantization (bits)	8	18	8	8	8	8	8
FPGA device	XCZU7EV	XC7VX485T	XC7K325T	XC7K325T	XC7VX690T	XC7VX690T	XC7VX690T
Technology (nm)	16	28	28	28	28	28	28
LUTs (K)	123/230	48.5/303	136.5/203	136.5/203	154.6/433	154.6/433	213.2/433
DSP blocks	146/1728	2304/2800	687/840	687/840	897/3600	897/3600	1737/3600
Block RAM 18K	326/624	141/2060	553/890	553/890	631/2940	631/2940	801/2940
Frequency (MHz)	200	200	200	200	180	180	160
Batch size	8	1	1	1	1	1	2
Throughput (GOP/S)	348.0	460	401	409	491	501	859
On-chip power(W)	3.3	4.8	7.8	7.8	8.42	8.42	12.61
Total power (W)	–	–	12.3	12.3	19.8	19.8	24.5
Efficiency (GOPs/W)	105.4/–	95.6/–	51.4/32.6	52.4/33.2	58.0/24.7	59.5/25.3	68.1/35.0

and hardware generating platform comprised an AMD R9-5950x CPU and 64 GB RAM. The time consumption for generating hardware varies with hardware parameters is shown in Fig. 16, where the compilation time increases with the increase in hardware parameters N_n and N_m. The compilation time also reflects the difficulty of hardware generation, which is often related to the factors such as parallelism and fan-out.

To ensure a fair comparison with previous works [6, 7, 9], the LUT, BRAM, and DSP utilization information reported by Vivado is utilized to evaluate the resource consumption of the accelerator. Both FPGA's power consumption and the power of the development board were measured. The FPGA's power consumption was reported by the power analyzer of Vivado, as in previous work [9]. The power consumption of the entire board was measured using the power meter plugged into the development platform, as in previous work [7]. The actual execution time of each layer was measured by using a software timer provided by the MicroBlaze MCU.

The design parameters for the two FPGA devices change accordingly based on their capacities. As per the model described in Sect. 4, the 325T FPGA device was evaluated using selected parameters, namely $N_n = 8$, $N_m = 12$, $N_r = 13$, and $N_b = 1$, which quantifies the accelerators' performance on resource-constrained devices. To assess the accelerator's performance on a device with larger capacity, performance tests were conducted using $N_n = 8$, $N_m = 16$, $N_r = 13$, $N_b = 1$, and $N_n = 8$, $N_m = 16$, $N_r = 13$, and $N_b = 2$.

Overall, the workflow of the accelerator design is shown in Fig. 17, which is mainly composed of the model preparation and hardware preparation. Once finishing training the CNN model, the parameters of BN are merged into the weights and bias, then quantization is conducted using the BRECQ method. For the hardware preparation, the optimal hardware parameters for the target FPGA device are generated using the performance and resource model. Then, the HLS compilation is conducted using the aforementioned hardware parameters. Finally, the IP cores are connected, and the hardware is generated using Vivado.

5.2 Performance evaluation of the hardware accelerator

The baseline YOLOv2 and YOLOv3 algorithms have been implemented on our proposed hardware accelerator. The theoretical execution time of each layer was used to measure the efficiency of the accelerator, and it is calculated by

$$t'_l = \frac{\#OP_l}{2 \times N_n \times N_m \times N_r \times N_b \times Freq}, \tag{32}$$

where the denominator represents the maximum throughput that the PE array can provide. The execution efficiency of the accelerator was calculated by $E_l = t'_l/t_l$, where t_l denotes the measured execution time of the l-th layer. For the YOLOv2 network, the theoretical execution time and actual execution time were also compared in Fig. 18. It can be observed that the actual execution time for most of the layers was close to the theoretical time except for six layers, i.e., the 1st, 4th, 7th, 21st, 23rd, and 24th layer. Two

problems led to the low efficiency of the first layer. Firstly, the first layer only has three input channels, resulting in inadequate utilization of the parallelism in the N_n direction. Secondly, according to the performance model, the calculation latency equals to $L_{CU} = K \times K' \times \lceil N/N_n \rceil = 3 \times 3 \times \lceil 3/8 \rceil = 9$, which is smaller than N_r. Consequently, N_r vectors each with a size of $1 \times 1 \times N_m$ are produced every L_{CU} clocks. However, it takes N_r clocks to write these vectors back to the DDR memory. Moreover, in contrast to convolutions with large filter sizes, 1×1 convolutions have relatively low reuse of the input feature, resulting in a low arithmetic intensity. According to the Roofline model, their performances are restricted by the memory bandwidth limitations. However, the execution time of these layers was very short, so the negative impact on the overall performance was negligible. The actual throughput of the proposed detection system equals to the ratio of the operation amount to the total measured time consumption, i.e.,

$$TPS = \frac{\sum_{l=1}^{L} \#OP_l}{\sum_{l=1}^{L} t_l}. \tag{33}$$

In this work, the achieved highest throughput of the accelerator was 914 GOP/s, which is achieved on the XC690T with the $N_n = 8$, $N_m = 16$, $N_r = 13$, and $N_b = 2$ and the frequency equals to 160 MHz. The power consumption of the FPGA device was 12.5 W, which was estimated using the Vivado development tool, and the power consumption of the whole board was 24.1 W. The performance of implementing YOLOv3 on the accelerator is shown in Fig. 19. The computation efficiency of ten layers are lower than 60%, and the reason for the low efficiency is the same as that of YOLOv2. Moreover, the execution time of these layers was also short, so the overall efficiency is relative high.

The resource consumption of each part of the accelerator is depicted in Fig. 20. Figure 20a indicates that the memory controller and AXI interconnect consume roughly 25% of the LUT resources, which poses challenges for FPGA's routing and timing optimization. Nevertheless, the model presented in Sect. 4.2 can still accurately predict LUT consumption because we have observed that the resource consumption of memory controller remains constant. Additionally, over 60% of the BRAM consumption in the accelerator is attributed to LocalBuffer. As shown in Fig. 20c, more than 90% of the DSP resources are utilized to implement the PE array. The resource consumption of the LUTs, BRAM, and DSPs are presented by small dots in Fig. 14, and it is apparent that they conform to the model presented in Sect. 4.2.

When implementing YOLOv2 algorithm, a total of $13 \times 13 \times 5 = 845$ candidate boxes were processed sequentially by the post-processing unit. As shown in Fig. 18, its time consumption equals to 0.38 ms, which is relatively low compared to the CNN computation. Moreover, the latency is proportional to the number of candidate boxes. The breakdown hardware consumption of the post-processing unit is shown in Table 2, which is composed by the three aforementioned units and one misc unit. The misc unit is mainly composed by channels and global memory ports. Compared to the approach of [10], the hardware resource consumption of our design is relatively low, which enables implementing the accelerator in some resource-constrained devices.

5.3 Comparison with state-of-the-art

The performance of several state-of-the-art hardware accelerators based on FPGA for object detection tasks is

Table 5 Performance evaluation between alternatives

Work	ACCESS 2021 [33]	ACCESS 2021 [33]	DSD2022 [31]	TCAD2020 [7]	Ours
Device	Intel NCS	Jetson nano	Intel GX1150	ZC706	Xilinx 325T
Architecture	ASIC	ARM+GPU	FPGA	ARM+FPGA	FPGA
Network	YOLOv3-Tiny	YOLOv3-Tiny	YOLOv3-Tiny	YOLO-Tiny	YOLOv3-Tiny
Resolution	448	448	–	–	416
Workload (GOP)	10.1	10.1	5.56	–	5.7
Precision	FP16	FP32	FP32	INT16	INT8
Frequency	700	1300	200	166	200
Power	1	10	–	9.4	7.8/12.3
Technology (nm)	–	–	20	28	28
Latency (ms)	55	40	27.5	–	15.2
Throughput (GOP/s)	183.6	252.5	202	201	401
Efficiency (GOP/W)	–/183.6	–/25.25	–/–	21.4	52.2/33.0

compared in Tables 3 and 4 depending on the networks used. To ensure fairness in the comparison, the performance results are measured and reported using a batch size of 1.

Table 3 presents a comparison between two reference accelerators that implement YOLOv2-related models and our proposed design. The design presented in [4] deployed the entire YOLOv2-Tiny neural network onto a single FPGA device, minimizing the need for external memory access. Furthermore, the computation of all convolution layers was organized as a deep pipeline and workload balance was employed to achieve a low inference latency of 27.78 ms with the input size of 1280×384. However, a limitation of this design is that the accelerator architecture is fixed and cannot be reprogrammed by software to implement larger detection models, such as standard YOLOv2 or YOLOv3. In contrast, the architecture proposed in this work is highly flexible and capable of executing convolution computations with arbitrary input and filter sizes. For instance, our design can implement the standard YOLOv2 model on the XC7325T device, which has even fewer logic and DSP resources than the device used in [4]. For the detection latency, our YOLOv2-Tiny implemented on XC325T achieved an end-to-end latency of 14.49ms with the input size of 416×416. In [9], a cross-layer dataflow strategy was introduced to optimize the frequency of external memory data access. However, this design implemented separated hardware units for the 1×1 and 3×3 convolution layers, leading to significant imbalance in hardware resource utilization. Consequently, the highest throughput achievable by the accelerator when executing the YOLOv2 model was limited to only 100.3 GOP/s. In contrast, the programmable convolution unit in our design can fully utilize as many as possible on-chip hardware resources to accelerate all the convolution layers in the neural network model, resulting a 5× throughput at the cost of 2.2× of on-chip DSP resources consumption.

In Table 4, our results are compared with those of two other FPGA accelerators that implement YOLOv3-related models. The design in [28] utilized the similarities between continuous inputs in video, and skipped the convolution operation of similar data between continuous inputs. Winograd algorithm was also used to optimize the 3×3 convolution in the YOLOv3-Tiny to further improve the throughput. Experimental results show that their accelerator achieved a throughput of 348 GOP/s on ZCU104 evaluation board. This work brings new highlights to the processing of continuous video input. However, a relative large batch size is required by that work to utilize the similarity between the input, which results in a large inference latency of 107.81 ms, and this is not suitable for implementing this accelerator on low latency scene. Compared with that work, our accelerator achieved an end-to-end latency of 15.2 ms on XC325T FPGA, which reduces 7× of latency. The design in [6] consumed 2304 DSPs of the XC7VX485T FPGA device and achieved a performance of 460.8 GOP/s. However, preprocessing, activation functions, pooling, and other miscellaneous operations were conducted by a low-performance MicroBlaze softcore. The sequential process of those functions likely affected the overall latency, but no information was reported about their overhead. Moreover, the model was represented by 16-bit fixed-point data, which consumed many DSP resources to perform the MAC operations and led to heavy bandwidth pressure on the off-chip bandwidth. In contrast, our design accelerated the entire detection procedure and achieved an end-to-end latency of only 15.2 ms on XC325T FPGA. Our accelerator achieved 87% of the throughput of the previous work, while using only 29.8% of the DSP resources. This was achieved by quantizing the model to 8-bit, which resulted in an mAP of 76.7% on the VOC data set.

To further evaluate the efficiency of our proposed design against accelerators implemented on different hardware platforms, we compare four technology alternatives—including ASIC, ARM+GPU, ARM+FPGA, and FPGA-only based object detection accelerators in Table 5. For ASIC-based designs, we select Intel NCS 2 [29], which achieves the lowest power consumption with a detection latency of 55 ms for YOLOv3-Tiny algorithm. The disadvantage of the ASIC design is that its function cannot upgrade to adapt for new detection algorithms, such YOLOv5, YOLOx, and new quantization schemes, such 4-bit/2-bit CNN models. For GPU-based designs, we select the Jetson nano development board [30], which can deliver a detection latency of 40 ms for YOLOv3-Tiny with 32-bit computation precision. Compared with that, our design on Xilinx 325T FPGA shows an advantage of 2x shorter latency with only 20% more power consumption. For FPGA-only design, we select the design implemented on a Intel GX1150 FPGA [31]. For YOLOv3-Tiny algorithm, the end-to-end latency is 27.5ms. Due to the 32-bit floating point data used for both the CNN computation and post-processing algorithms, this design consumes more hardware resource and result in inferior detection latency. For ARM+FPGA implementation, we compare the design presented in [7], which employed the fast Fourier transformation (FFT) algorithm to reduce the number of multiplications required for convolution computation. However, to maintain the same computation precision, the input operands for the complex multiplications utilized in the FFT were quantized into 16-bit width, resulting in roughly 2× external memory bandwidth compared to an 8-bit quantized convolution approach. Consequently, its performance was limited by the external memory bandwidth and its highest achievable throughput was only

$201.1/401 = 50.1\%$ that of our proposed accelerator when implemented on a device with similar resource, i.e., XC485T FPGA. Moreover, it only accelerated the CNN computation, while the post-processing was conducted on the CPU. However, the previous works [11, 32] show that the post-processing is crucial for the total latency of the detection.

We also evaluate the power efficiency of the accelerators with different technologies. It can be found that the ASIC-based accelerator Intel NCS2 achieved the highest power efficiency, because the hardware architecture of the ASIC-based accelerator has been carefully optimized. Compared to our implementation on 325T FPGA, the work in ISCAS2021 [4] achieved higher power efficiency. This is because the streaming accelerator has a low DDR access frequency, which reduces the power consumption of the DDR access.

Despite the improvements achieved by our proposed design, there are still some limitations that constrain further performance enhancement. For instance, when the batch size is equal to one, the throughput of the accelerator applied to the XC690T device is only 1.25 times greater than that of the XC325T device, while DSP utilization is only 25%. This limitation can be attributed to two main factors. First, as the degree of parallelism increases, more hardware resources are required. This results in an increase in difficult in FPGA's place and routing and a subsequent reduction in the operating frequency of the accelerator. For example, the parameters of the accelerator on the 690T were increased from $Nm = 12$ to $Nm = 16$, the operating frequency decreased from 200MHz to 180 MHz. Secondly, as shown in Figs. 18 and 19, performance is limited by the bandwidth of external memory when performing 1×1 convolution. As such, further improvement in throughput can only be achieved by increasing the batch size to enhance computational intensity.

6 Conclusion

This paper presents an efficient FPGA accelerator design for YOLO series detection algorithms. The design utilizes multiple dimensions of parallelism to accelerate the convolution layers of the network model, and employs multi-port parallel line-buffer-like on-chip data cache to minimize the external memory bandwidth utilization, resulting in improved detection performance when compared to state-of-the-art FPGA accelerators. Moreover, the post-processing algorithms of the YOLO models are implemented by dedicated hardware circuits in the proposed accelerator, which significantly reduced the end-to-end detection latency when compared to GPU and CPU-based implementation approaches. The combination of algorithm and hardware optimization enabled the accelerator to achieve a 4× of throughput when implementing the same YOLOv2 algorithm on a low-end 325T FPGA and achieves a throughput of 859 GOPs/s when implementing YOLOv3 on Xilinx 690T FPGA.

In future work, we plan to utilize more advanced schemes to quantize the convolution with even lower bit width, such as 4-bit or 2-bit quantization, to further reduce the hardware resource cost and the bandwidth utilization. We also plan to implement other object detectors, such as faster-RCNN and FPN, to further evaluate the efficiency of the proposed design for two-stage-based detection algorithms.

Acknowledgements This work was supported by Beijing Natural Science Foundation under Grant No. 4202063, National Key Research and Development Program of China under Grant No. 2019YFB2204200.

Availability of data and materials All data generated or analyzed during this study are included in this published article.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Carranza-García M, Lara-Benítez P, García-Gutiérrez J, Riquelme JC (2021) Enhancing object detection for autonomous driving by optimizing anchor generation and addressing class imbalance. Neurocomputing 449:229–244. https://doi.org/10.1016/j.neucom.2021.04.001
2. Nguyen EH, Yang H, Deng R, Lu Y, Zhu Z, Roland JT, Lu L, Landman BA, Fogo AB, Huo Y (2022) Circle representation for medical object detection. IEEE Trans Med Imaging 41(3):746–754. https://doi.org/10.1109/TMI.2021.3122835
3. Angelo TD, Mendes M, Keller B, Ferreira R, Delabrida S, Rabelo R, Azpurua H, Bianchi A (2019) Deep learning-based object detection for digital inspection in the mining industry. In: 2019 18th ieee international conference on machine learning and applications (ICMLA), pp 633–640. https://doi.org/10.1109/ICMLA.2019.00116
4. Zhang J, Cheng L, Li C, Li Y, He G, Xu N, Lian Y (2021) A low-latency FPGA implementation for real-time object detection. In: 2021 IEEE international symposium on circuits and systems (ISCAS), pp 1–5
5. Nguyen DT, Nguyen TN, Kim H, Lee H-J (2019) A high-throughput and power-efficient FPGA implementation of YOLO CNN for object detection. IEEE Trans Very Large Scale Integr (VLSI) Syst 27(8):1861–1873. https://doi.org/10.1109/TVLSI.2019.2905242
6. Ahmad A, Pasha MA, Raza GJ (2020) Accelerating tiny YOLOv3 using FPGA-based hardware/software co-design. In: 2020 IEEE international symposium on circuits and systems (ISCAS), pp 1–5. https://doi.org/10.1109/ISCAS45731.2020.9180843

7. Liang Y, Lu L, Xiao Q, Yan S (2020) Evaluating fast algorithms for convolutional neural networks on FPGAs. IEEE Trans Comput Aided Des Integr Circuits Syst 39(4):857–870
8. Capotondi A, Rusci M, Fariselli M, Benini L (2020) CMix-NN: mixed low-precision CNN library for memory-constrained edge devices. IEEE Trans Circuits Syst II Express Briefs 67(5):871–875. https://doi.org/10.1109/TCSII.2020.2983648
9. Zhang Z, Mahmud MAP, Kouzani AZ (2022) Resource-constrained FPGA implementation of YOLOv2. Neural Comput Appl 34(19):16989–17006. https://doi.org/10.1007/s00521-022-07351-w
10. Anupreetham A, Ibrahim M, Hall M, Boutros A, Kuzhively A, Mohanty A, Nurvitadhi E, Betz V, Cao Y, Seo J-s (2021) End-to-end FPGA-based object detection using pipelined CNN and non-maximum suppression. In: 2021 31st international conference on field-programmable logic and applications (FPL), pp 76–82. https://doi.org/10.1109/FPL53798.2021.00021. ISSN: 1946-1488
11. Shi M, Ouyang P, Yin S, Liu L, Wei S (2019) A fast and power-efficient hardware architecture for non-maximum suppression. IEEE Trans Circuits Syst II Express Briefs 66(11):1870–1874. https://doi.org/10.1109/TCSII.2019.2893527
12. Redmon J, Farhadi A (2017) YOLO9000: better, faster, stronger. In: 2017 IEEE conference on computer vision and pattern recognition (CVPR), pp 6517–6525. https://doi.org/10.1109/CVPR.2017.690
13. Li Y, Gong R, Tan X, Yang Y, Hu P, Zhang Q, Yu F, Wang W, Gu S (2021) BRECQ: pushing the limit of post-training quantization by block reconstruction. arXiv. arXiv:2102.05426 [cs]. https://doi.org/10.48550/arXiv.2102.05426. Accessed 18 Apr 2023
14. Nagel M, Amjad RA, Baalen MV, Louizos C, Blankevoort T (2020) Up or down? adaptive rounding for post-training quantization. In: Hal I, Aarti S (eds.) Proceedings of the 37th international conference on machine learning, vol 119. PMLR, pp 7197–7206. https://proceedings.mlr.press/v119/nagel20a.html
15. Gysel P, Pimentel J, Motamedi M, Ghiasi S (2018) Ristretto: a framework for empirical study of resource-efficient inference in convolutional neural networks. IEEE Trans Neural Netw Learn Syst 29(11):5784–5789. https://doi.org/10.1109/TNNLS.2018.2808319
16. Wang D, Xu K, Jiang D (2017) PipeCNN: an opencl-based open-source FPGA accelerator for convolution neural networks. In: 2017 international conference on field programmable technology (ICFPT), pp 279–282
17. Véstias M, Duarte RP, Sousa JTd, Neto H (2017) Parallel dot-products for deep learning on FPGA. In: 2017 27th international conference on field programmable logic and applications (FPL), pp 1–4. https://doi.org/10.23919/FPL.2017.8056863
18. Fu Y, Wu E, Sirasao A, Attia S, Khan K, Wittig R (2016) Deep learning with int8 optimization on xilinx devices
19. Xilinx: UltraScale architecture and product data sheet: overview (2020). https://www.xilinx.com/support/documentation/data_sheets/ds890-ultrascale-overview.pdf
20. Guo L, Lau J, Chi Y, Wang J, Yu CH, Chen Z, Zhang Z, Cong J (2020) Analysis and optimization of the implicit broadcasts in FPGA HLS to improve maximum frequency. In: 2020 57th ACM/IEEE design automation conference (DAC), pp 1–6. https://doi.org/10.1109/DAC18072.2020.9218718
21. Wang D, Xu K, Guo J, Ghiasi S (2020) DSP-efficient hardware acceleration of convolutional neural network inference on FPGAs. IEEE Trans Comput Aided Des Integr Circuits Syst 39(12):4867–4880
22. Obeidat F, Klenke R (2011) Introducing MicroBlaze as an infrastructure for performance modeling. In: 2011 IEEE international conference on microelectronic systems education, pp 90–93. https://doi.org/10.1109/MSE.2011.5937101
23. Xu M, Yao H, Huan X (2012) Performance test of dual-core processor system based on NIOS II. In: 2012 IEEE symposium on electrical & electronics engineering (EEESYM), pp 82–85. https://doi.org/10.1109/EEESym.2012.6258593
24. Williams S, Waterman A, Patterson D (2009) Roofline: an insightful visual performance model for multicore architectures. Commun ACM 52(4):65–76. https://doi.org/10.1145/1498765.1498785
25. Zhang C, Li P, Sun G, Guan Y, Xiao B, Cong J (2015) Optimizing FPGA-based accelerator design for deep convolutional neural networks. In: Proceedings of the 2015 ACM/SIGDA international symposium on field-programmable gate arrays, pp 161–170. Association for Computing Machinery, Monterey California USA. https://doi.org/10.1145/2684746.2689060
26. Chen K, Wang J, Pang J, Cao Y, Xiong Y, Li X, Sun S, Feng W, Liu Z, Xu J, Zhang Z, Cheng D, Zhu C, Cheng T, Zhao Q, Li B, Lu X, Zhu R, Wu Y, Dai J, Wang J, Shi J, Ouyang W, Loy CC, Lin D (2019) MMDetection: Open mmlab detection toolbox and benchmark. arXiv preprint arXiv:1906.07155
27. Everingham M, Van Gool L, Williams CKI, Winn J, Zisserman A (2010) The PASCAL visual object classes (VOC) challenge. Int J Comput Vis 88(2):303–338. https://doi.org/10.1007/s11263-009-0275-4
28. Li S, Wang Q, Jiang J, Sheng W, Jing N, Mao Z (2022) An efficient CNN accelerator using inter-frame data reuse of videos on FPGAs. IEEE Trans Very Large Scale Integr (VLSI) Syst 30(11):1587–1600. https://doi.org/10.1109/TVLSI.2022.3151788
29. Intel neural compute stick 2. https://www.intel.com/content/www/cn/zh/developer/articles/tool/neural-compute-stick.html Accessed 10 May 2023
30. Jetson nano developer kit for AI and robotics | NVIDIA. https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-nano/ Accessed 10 May 2023
31. Herrmann V, Knapheide J, Steinert F, Stabernack B (2022) A YOLO v3-tiny FPGA architecture using a reconfigurable hardware accelerator for real-time region of interest detection. In: 2022 25th Euromicro conference on digital system design (DSD), pp 84–92. https://doi.org/10.1109/DSD57027.2022.00021. ISSN: 2771-2508
32. Zhang H, Wu W, Ma Y, Wang Z (2020) Efficient hardware post processing of anchor-based object detection on FPGA. In: 2020 IEEE computer society annual symposium on VLSI (ISVLSI). IEEE, Limassol, Cyprus, pp 580–585. https://doi.org/10.1109/ISVLSI49217.2020.00089. https://ieeexplore.ieee.org/document/9155076/ Accessed 15 Nov 2022
33. Adiono T, Putra A, Sutisna N, Syafalni I, Mulyawan R (2021) Low latency YOLOv3-Tiny accelerator for low-cost FPGA using general matrix multiplication principle. IEEE Access 9:141890–141913. https://doi.org/10.1109/ACCESS.2021.3120629

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.